

# Wstęp do uczenia maszynowego

Jak sprawdzić, czy model naprawdę się nauczył?

---

Mikołaj Słupiński

26 września 2026

Olimpiada Sztucznej Inteligencji — szkolenia

1. Model, dane, uczenie — przypomnienie
2. Najbliżsi sąsiedzi (kNN)
3. Podział danych: trening / walidacja / test
4. Przeuczenie, niedouczenie, obciążenie i wariancja
5. Walidacja krzyżowa
6. Metryki: po czym poznać dobry model?
7. Dobór hiperparametrów: grid search, random search, Optuna
8. Drzewa decyzyjne
9. Zespoły modeli: bagging, lasy losowe, voting, boosting, XGBoost

## Pytanie przewodnie

Skąd wiemy, że model *nauczył się* reguły, a nie *wykuł na pamięć* przykładów?

## Model, dane, uczenie

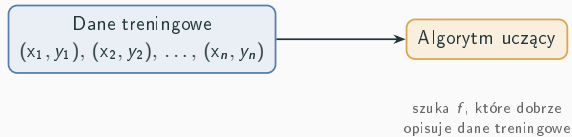
---

# Uczenie nadzorowane w jednym obrazku

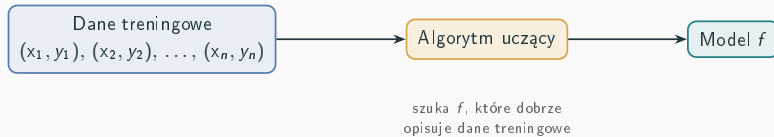
Dane treningowe

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

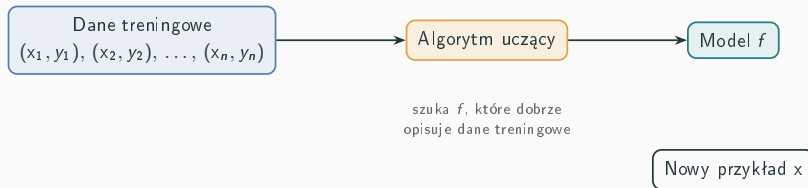
# Uczenie nadzorowane w jednym obrazku



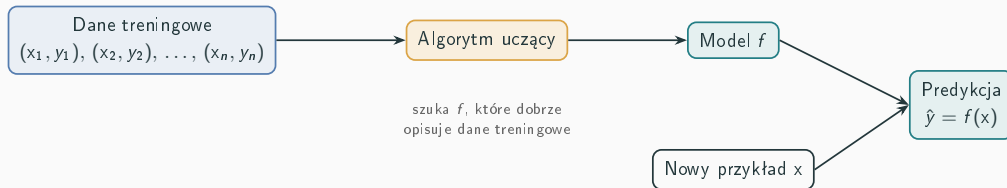
# Uczenie nadzorowane w jednym obrazku



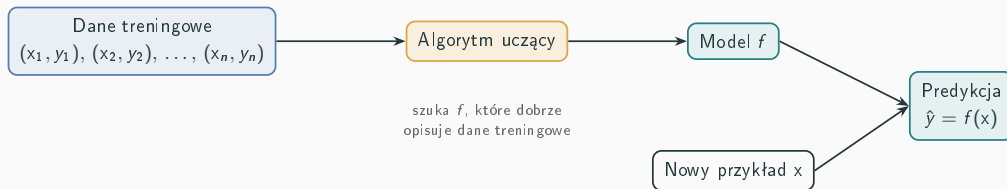
# Uczenie nadzorowane w jednym obrazku



# Uczenie nadzorowane w jednym obrazku



# Uczenie nadzorowane w jednym obrazku



**Klasyfikacja:**  $y$  to wartość dyskretna (klasa, etykieta)

- spam / nie spam
- chory / zdrowy
- kot / pies / królik

**Regresja:**  $y$  to wartość ciągła (liczba)

- cena mieszkania
- temperatura jutro o 12:00
- czas przejazdu

# Model może się nauczyć albo wykuć na pamięć

**Uczeń A** rozwiązał 200 zadań z podręcznika i zna każde na pamięć.

- na tych samych zadaniach: 100 %
- na nowym zadaniu z matury: ?

**Uczeń B** rozwiązał 200 zadań i zrozumiał metodę.

- na tych samych zadaniach: 95 %
- na nowym zadaniu z matury: też około 95 %

# Model może się nauczyć albo wykuć na pamięć

**Uczeń A** rozwiązał 200 zadań z podręcznika i zna każde na pamięć.

- na tych samych zadaniach: 100 %
- na nowym zadaniu z matury: ?

**Uczeń B** rozwiązał 200 zadań i zrozumiał metodę.

- na tych samych zadaniach: 95 %
- na nowym zadaniu z matury: też około 95 %

## **W uczeniu maszynowym**

Interesuje nas wynik na **nowych** danych (*generalizacja*), a nie na tych, które model już widział.

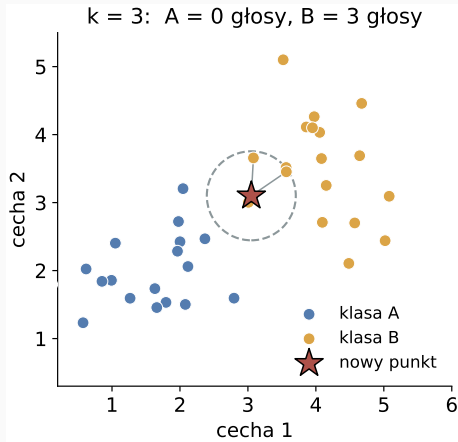
Dzisiejszy wykład to zestaw narzędzi, które pozwalają:

- uczciwie **zmierzyć** generalizację,
- **dobrać** model i jego ustawienia,
- **rozpoznać**, kiedy model kuje na pamięć.

## Najbliżsi sąsiedzi (kNN)

---

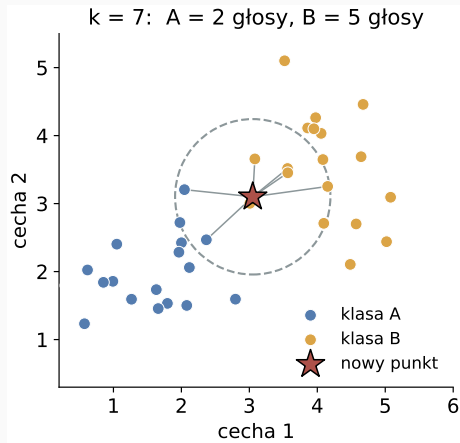
# Kto z kim przystaje, takim się staje



## Algorytm $k$ najbliższych sąsiadów (kNN)

1. Zapamiętaj wszystkie przykłady treningowe.
2. Dla nowego punktu  $x$  policz odległość do każdego z nich.
3. Wybierz  $k$  najbliższych.
4. Klasa = głosowanie większościowe sąsiadów.

# Kto z kim przystaje, takim się staje

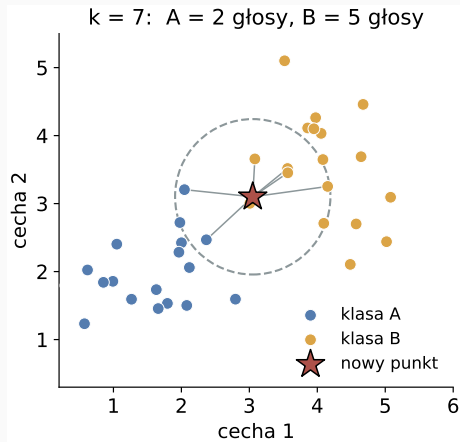


## Algorytm $k$ najbliższych sąsiadów (kNN)

1. Zapamiętaj wszystkie przykłady treningowe.
2. Dla nowego punktu  $x$  policz odległość do każdego z nich.
3. Wybierz  $k$  najbliższych.
4. Klasa = głosowanie większościowe sąsiadów.

Ten sam punkt, inne  $k$ : 3:0 dla  $k = 3$ , 5:2 dla  $k = 7$ . Wybór  $k$  to **nasza decyzja**.

# Kto z kim przystaje, takim się staje



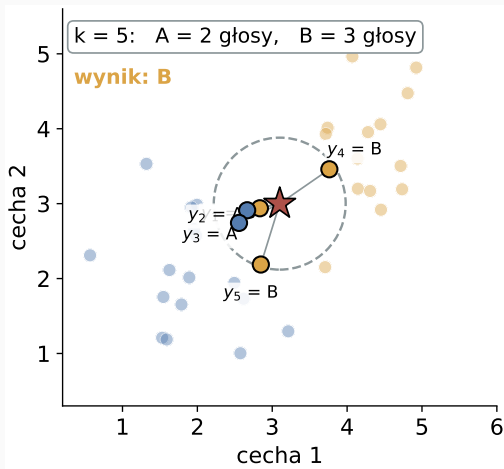
## Algorytm $k$ najbliższych sąsiadów (kNN)

1. Zapamiętaj wszystkie przykłady treningowe.
2. Dla nowego punktu  $x$  policz odległość do każdego z nich.
3. Wybierz  $k$  najbliższych.
4. Klasa = głosowanie większościowe sąsiadów.

Ten sam punkt, inne  $k$ : 3:0 dla  $k = 3$ , 5:2 dla  $k = 7$ . Wybór  $k$  to **nasza decyzja**.

Brak fazy uczenia (*lazy learning*): cała praca dzieje się przy predykcji.

## kNN wzorem: klasyfikacja

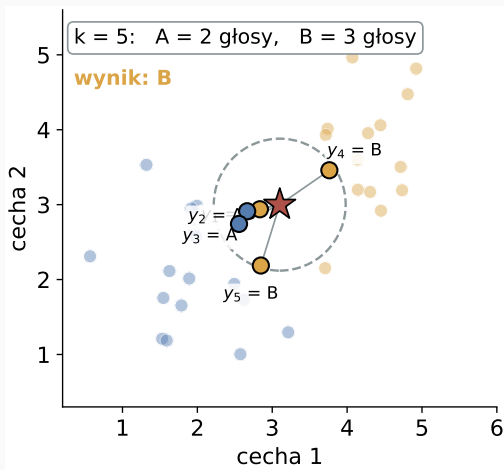


Sąsiedzi **głosują**; wygrywa klasa z największą liczbą głosów:

$$\hat{y} = \arg \max_{c \in \mathcal{Y}} \sum_{i \in N_k(x)} \mathbb{I}(y_i = c)$$

- $N_k(x)$ : indeksy  $k$  najbliższych sąsiadów punktu  $x$
- $\mathbb{I}(y_i = c)$ : 1, gdy sąsiad  $i$  ma klasę  $c$ , inaczej 0 — czyli jeden głos
- suma = liczba głosów na klasę  $c$

## kNN wzorem: klasyfikacja



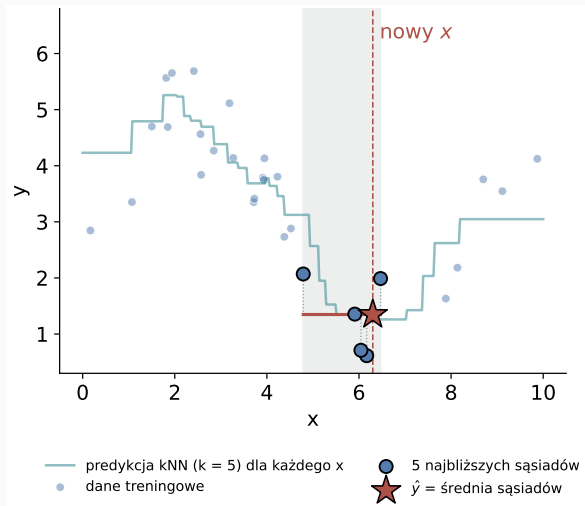
Sąsiedzi **głosują**; wygrywa klasa z największą liczbą głosów:

$$\hat{y} = \arg \max_{c \in \mathcal{Y}} \sum_{i \in N_k(x)} \mathbb{I}(y_i = c)$$

- $N_k(x)$ : indeksy  $k$  najbliższych sąsiadów punktu  $x$
- $\mathbb{I}(y_i = c)$ : 1, gdy sąsiad  $i$  ma klasę  $c$ , inaczej 0 — czyli jeden głos
- suma = liczba głosów na klasę  $c$

Na rysunku:  $\sum \mathbb{I}(y_i = A) = 2$ ,  
 $\sum \mathbb{I}(y_i = B) = 3$ , więc  $\hat{y} = B$ .

# kNN wzorem: regresja

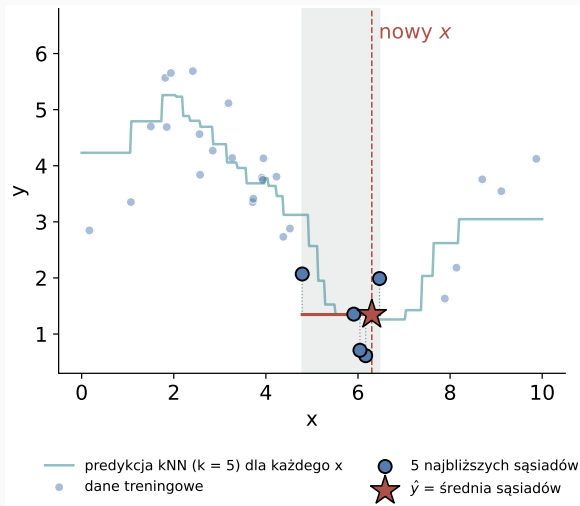


Gdy  $y$  jest liczbą, zamiast głosować  
**uśredniamy:**

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

- bierzemy  $k$  punktów o najbliższym  $x$  (szary pas)
- odpowiedź to średnia ich  $y$  (czerwony odcinek)

# kNN wzorem: regresja



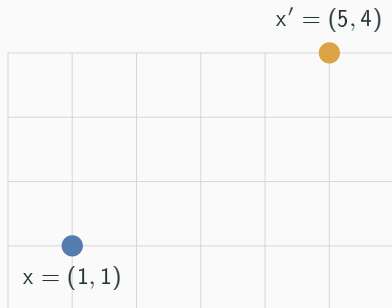
Gdy  $y$  jest liczbą, zamiast głosować  
**uśredniamy:**

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

- bierzemy  $k$  punktów o najbliższym  $x$  (szary pas)
- odpowiedź to średnia ich  $y$  (czerwony odcinek)

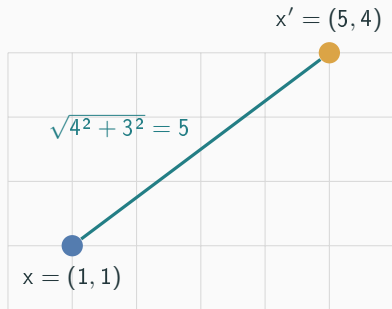
Ta sama reguła zastosowana dla każdego  $x$  daje schodkową krzywą: predykcja zmienia się tylko wtedy, gdy zmienia się skład sąsiadów.

## Co znaczy „najbliższy”?



Dwa punkty w przestrzeni cech. Jak daleko są od siebie?

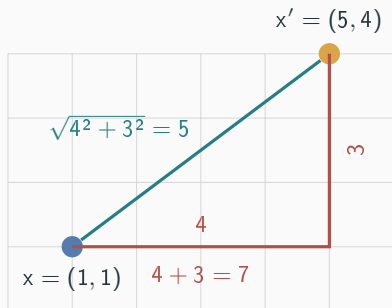
# Co znaczy „najbliższy”?



**Odległość euklidesowa (L2):** długość prostej

$$d_{\text{eukl}}(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2}$$

# Co znaczy „najbliższy”?



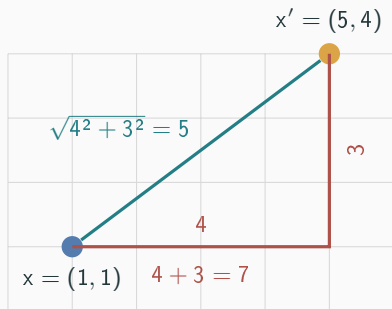
**Odległość euklidesowa (L2):** długość prostej

$$d_{\text{eukl}}(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2}$$

**Odległość Manhattan (L1):** droga po ulicach siatki

$$d_{\text{manh}}(x, x') = \sum_{j=1}^d |x_j - x'_j|$$

# Co znaczy „najbliższy”?



**Odległość euklidesowa (L2):** długość prostej

$$d_{\text{eukl}}(x, x') = \sqrt{\sum_{j=1}^d (x_j - x'_j)^2}$$

**Odległość Manhattan (L1):** droga po ulicach siatki

$$d_{\text{manh}}(x, x') = \sum_{j=1}^d |x_j - x'_j|$$

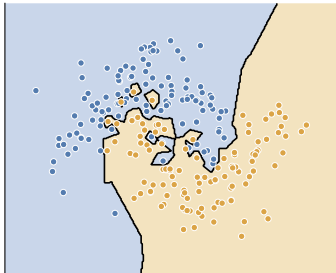
**Odległość Minkowskiego:** jedna rodzina, parametr  $p$

$$d_p(x, x') = \left( \sum_{j=1}^d |x_j - x'_j|^p \right)^{1/p}$$

$p = 2$  daje euklidesową,  $p = 1$  Manhattan. Wybór metryki to kolejny **hiperparametr** kNN.

# Jak $k$ zmienia granicę decyzyjną

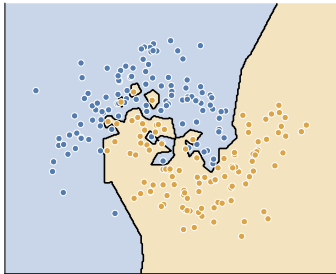
$k = 1$  (trafność na treningu: 100%)



$k = 1$ : każdy punkt, także szum, ma własną „wyspę”. Na treningu 100 %, ale granica jest chaotyczna.

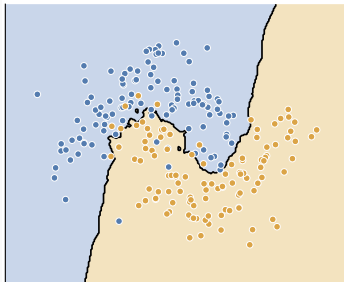
# Jak $k$ zmienia granicę decyzyjną

$k = 1$  (trafność na treningu: 100%)



$k = 1$ : każdy punkt, także szum, ma własną „wyspę”. Na treningu 100%, ale granica jest chaotyczna.

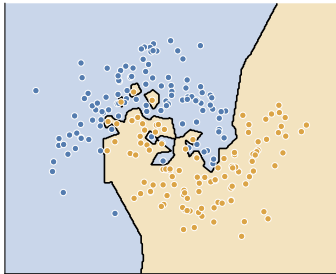
$k = 7$  (trafność na treningu: 95%)



$k = 7$ : granica gładka, ale nadal podąża za kształtem danych.

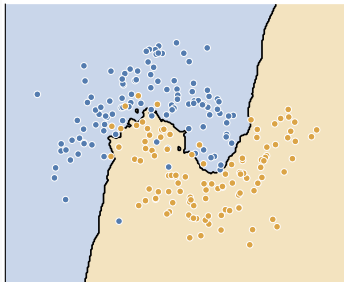
# Jak $k$ zmienia granicę decyzyjną

$k = 1$  (trafność na treningu: 100%)



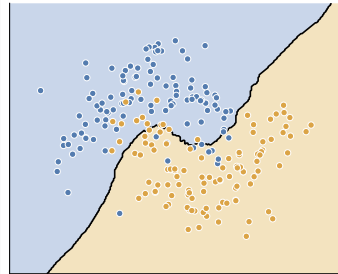
$k = 1$ : każdy punkt, także szum, ma własną „wyspę”. Na treningu 100%, ale granica jest chaotyczna.

$k = 7$  (trafność na treningu: 95%)



$k = 7$ : granica gładka, ale nadal podąża za kształtem danych.

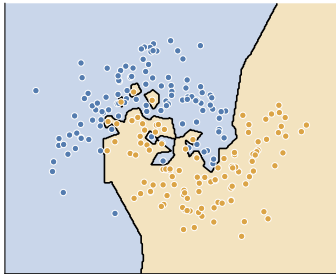
$k = 51$  (trafność na treningu: 90%)



$k = 51$ : granica prawie prosta; model ignoruje szczegóły, także te prawdziwe.

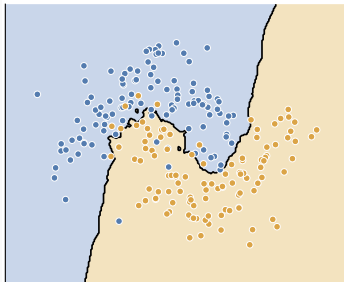
# Jak $k$ zmienia granicę decyzyjną

$k = 1$  (trafność na treningu: 100%)



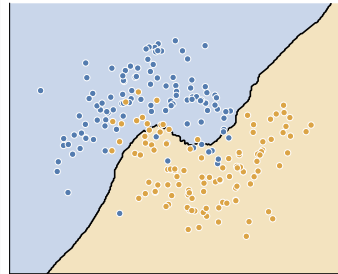
$k = 1$ : każdy punkt, także szum, ma własną „wyspę”. Na treningu 100%, ale granica jest chaotyczna.

$k = 7$  (trafność na treningu: 95%)



$k = 7$ : granica gładka, ale nadal podąża za kształtem danych.

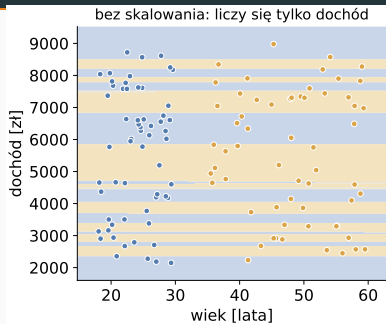
$k = 51$  (trafność na treningu: 90%)



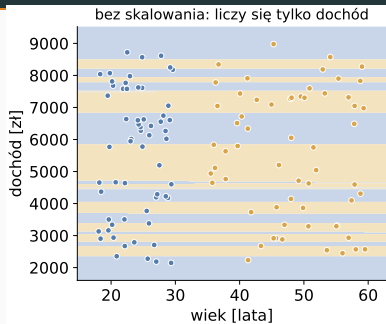
$k = 51$ : granica prawie prosta; model ignoruje szczegóły, także te prawdziwe.

**Które  $k$  jest najlepsze? Do tego wrócimy — najpierw musimy umieć to zmierzyć.**

# Pułapka: cechy w różnych jednostkach

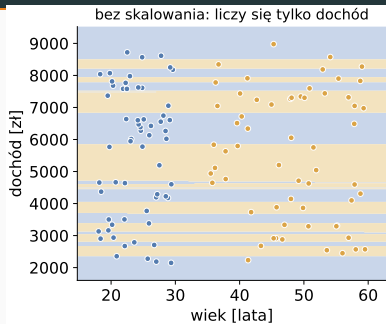


# Pułapka: cechy w różnych jednostkach



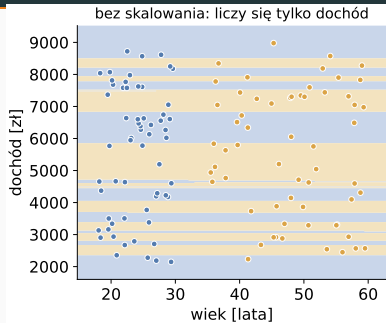
Różnica wieku o 20 lat to 20, różnica dochodu o 20 zł to też 20. Dochód (w tysiącach) „zagłusza” wiek.

# Pułapka: cechy w różnych jednostkach

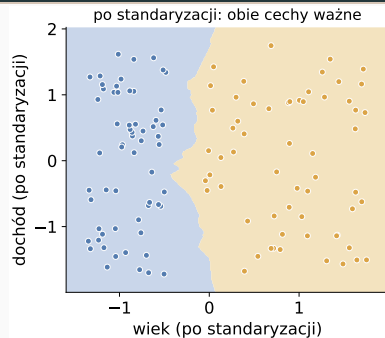


standaryzacja

# Pułapka: cechy w różnych jednostkach



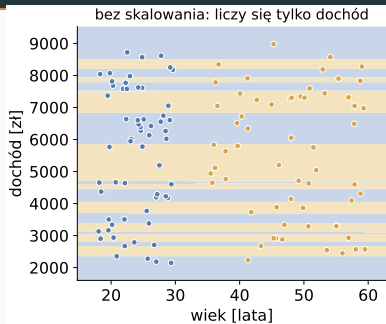
→  
standaryzacja



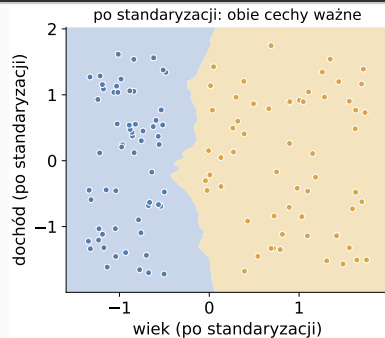
$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

każda cecha: średnia 0,  
odchylenie 1

# Pułapka: cechy w różnych jednostkach



→  
standaryzacja



$$z_j = \frac{x_j - \mu_j}{\sigma_j} \quad \text{każda cecha: średnia 0, odchylenie 1}$$

Średnią  $\mu_j$  i odchylenie  $\sigma_j$  liczymy **tylko** na zbiorze treningowym; tym samym przekształceniem skalujemy walidację i test.

# kNN w kodzie

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)           # statystyki tylko z treningu
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(scaler.transform(X_train), y_train)
y_pred = knn.predict(scaler.transform(X_new))
```

- fit na skalerze uczy się  $\mu_j$  i  $\sigma_j$  z treningu; transform stosuje je do dowolnych danych.
- fit na kNN tylko zapamiętuje punkty; cała praca dzieje się w predict.
- n\_neighbors to  $k$ ; metric i p wybierają odległość.

## Zalety

- prosty do zrozumienia i implementacji
- brak fazy uczenia
- potrafi modelować skomplikowane granice
- działa dla klasyfikacji i regresji

## Wady

- predykcja kosztuje  $O(n \cdot d)$  — trzeba przejrzeć cały zbiór
- trzeba przechowywać wszystkie dane
- wymaga skalowania cech
- w wielu wymiarach „wszyscy są daleko” (przekleństwo wymiarowości)

Podział danych: trening,  
walidacja, test

---

# Sprawdzian z zadań, które już widziałeś, nic nie mówi



# Sprawdzian z zadań, które już widziałeś, nic nie mówi



# Sprawdzian z zadań, które już widziałeś, nic nie mówi



## Zasada

Zbiór testowy udaje „przyszłość”. Każde zerknięcie na niego podczas budowy modelu sprawia, że wynik na teście przestaje być uczciwy.

# Zbiór treningowy: tu model się uczy

zbiór treningowy (60 %)

walidacyjny  
(20 %)

testowy  
(20 %)

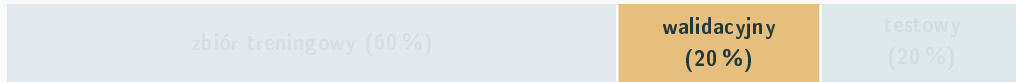
**Kto go widzi?** algorytm uczący.

**Co z nim robimy?** dopasowujemy **parametry** modelu: kNN zapamiętuje punkty, wielomian dobiera współczynniki, drzewo wybiera progi.

**Ile razy używamy?** wiele — przy każdym wariancie modelu od nowa.

**Co mierzy wynik na nim?** jak dobrze model pasuje do tego, co *już widział*. Dla  $k = 1$  to zawsze 100 %, więc niewiele mówi.

# Zbiór walidacyjny: tu wybieramy



**Kto go widzi?** my, gdy porównujemy warianty. Algorytm uczący — nie.

**Co z nim robimy?** odpowiadamy na pytania „ $k = 3$  czy  $k = 7$ ?”, „kNN czy drzewo?”, „które cechy?”.

**Ile razy używamy?** wiele — raz na każdy wariant.

**Co mierzy wynik na nim?** generalizację na nowe dane, ale „skażoną” naszymi wyborami: wybieramy przecież wariant, który wypadł tu najlepiej.

# Zbiór testowy: tu oceniamy — raz

zbiór treningowy (60 %)

walidacyjny  
(20 %)

testowy  
(20 %)

**Kto go widzi?** nikt, aż do samego końca.

**Co z nim robimy?** po wybraniu *jednego* finalnego modelu liczymy na nim końcowy wynik.

**Ile razy używamy?** raz.

**Co mierzy wynik na nim?** uczciwą generalizację: dokładnie to, co zobaczymy na danych z „prawdziwego świata”.

# Zbiór testowy: tu oceniamy — raz

zbiór treningowy (60 %)

walidacyjny  
(20 %)

testowy  
(20 %)

**Kto go widzi?** nikt, aż do samego końca.

**Co z nim robimy?** po wybraniu *jednego* finalnego modelu liczymy na nim końcowy wynik.

**Ile razy używamy?** raz.

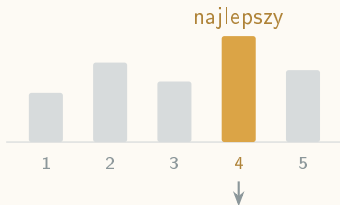
**Co mierzy wynik na nim?** uczciwą generalizację: dokładnie to, co zobaczymy na danych z „prawdziwego świata”.

Jeśli po zobaczeniu wyniku na teście wrócimy i coś poprawimy, test staje się drugim zbiorem walidacyjnym. Uczciwej oceny już nie odzyskamy.

# Dwie pułapki

## Wygrywa też szczęście

Wiele prób na tej samej walidacji



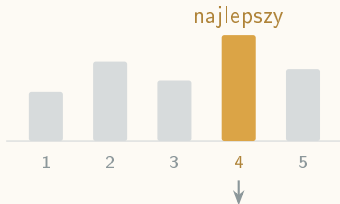
Oceń na teście

Walidacja wybiera. Test ocenia.

# Dwie pułapki

## Wygrywa też szczęście

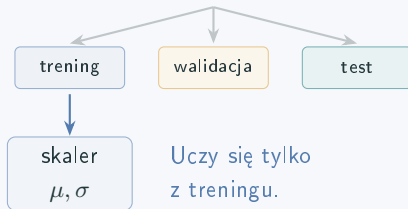
Wiele prób na tej samej walidacji



Walidacja wybiera. Test ocenia.

## Wyciek danych

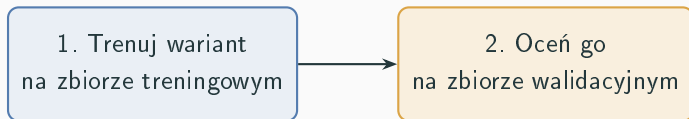
Najpierw podział



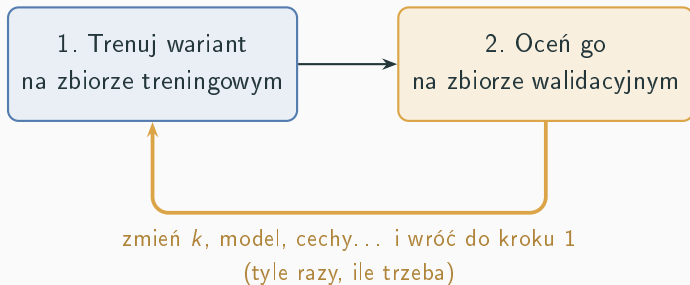
To samo przekształcenie dla treningu, walidacji i testu.

1. Trenuj wariant  
na zbiorze treningowym

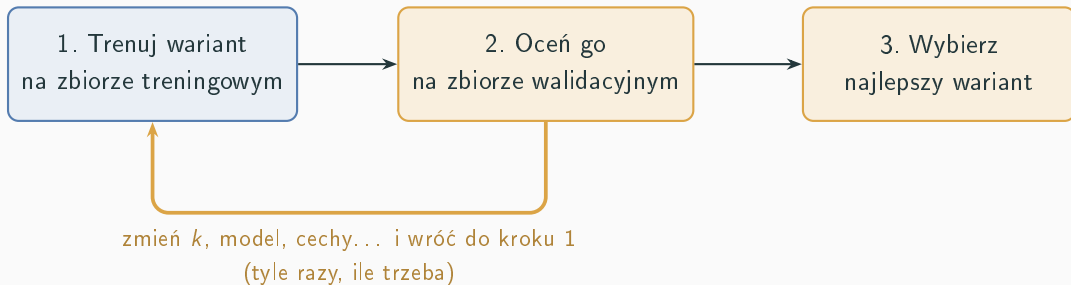
# Typowy przebieg pracy



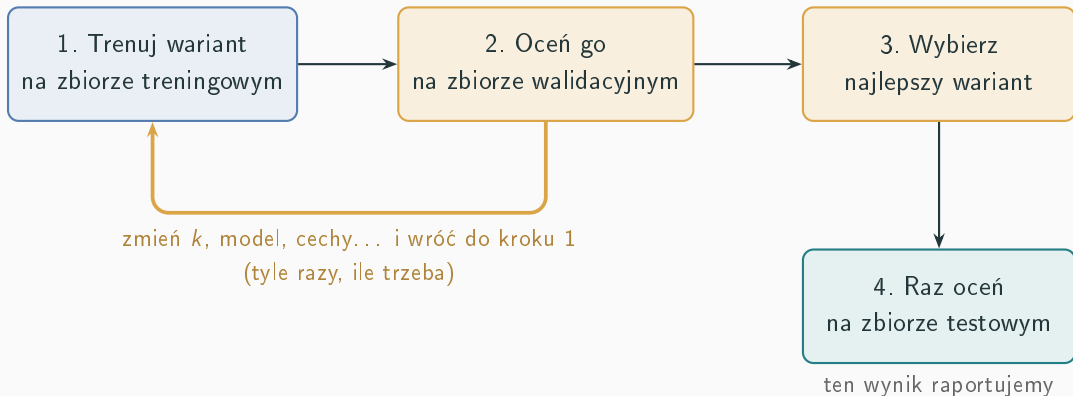
# Typowy przebieg pracy



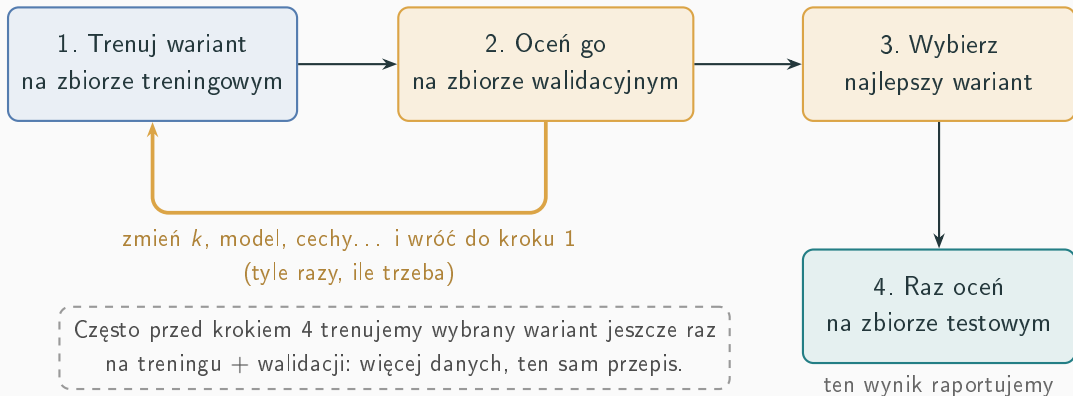
# Typowy przebieg pracy



# Typowy przebieg pracy



# Typowy przebieg pracy



# Podział danych w kodzie

```
from sklearn.model_selection import train_test_split

# najpierw odkładamy test (20 %) ...
X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# ... potem resztę dzielimy na trening (60 %) i walidację (20 %)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)
```

random\_state: ustalone ziarno losowania.

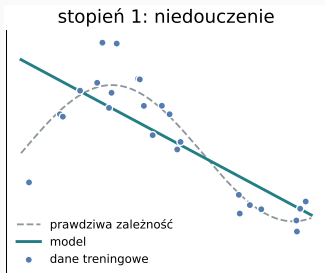
Podział jest losowy, ale *powtarzalny*: za tydzień dostaniemy te same zbiory.

stratify=y (stratyfikacja): każdy zbiór ma te same proporcje klas co całość. Gdy klasa dodatnia to 5 %, bez stratyfikacji mały zbiór walidacyjny może nie dostać jej wcale.

Przeuczenie, niedouczenie i  
kompromis  
obciążenie–wariancja

---

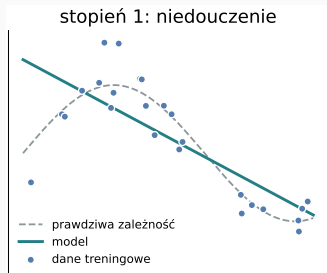
# Trzy modele, te same dane



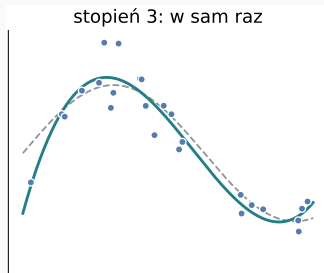
## Niedouczenie (underfitting):

model za prosty, nie łapie nawet głównego wzorca. Zły na treningu i na nowych danych.

# Trzy modele, te same dane

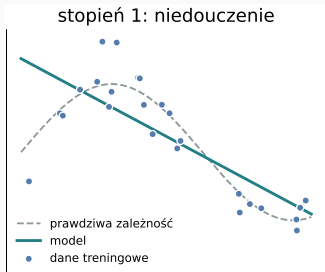


**Niedouczenie (underfitting):**  
model za prosty, nie łapie nawet  
głównego wzorca. Zły na treningu  
i na nowych danych.

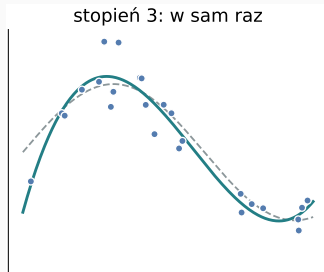


**W sam raz:** model odtwarza  
wzorec, ignoruje szum.

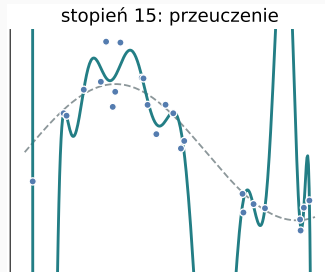
# Trzy modele, te same dane



**Niedouczenie (underfitting):** model za prosty, nie łapie nawet głównego wzorca. Zły na treningu i na nowych danych.

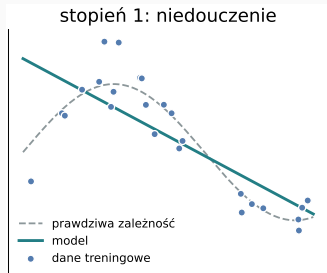


**W sam raz:** model odtwarza wzorec, ignoruje szum.

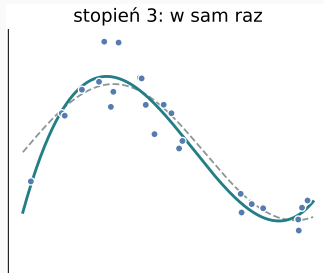


**Przeuczenie (overfitting):** model przechodzi przez każdy punkt, także przez szum. Świetny na treningu, fatalny na nowych danych.

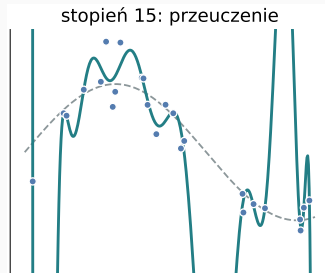
# Trzy modele, te same dane



**Niedouczenie** (underfitting): model za prosty, nie łapie nawet głównego wzorca. Zły na treningu i na nowych danych.



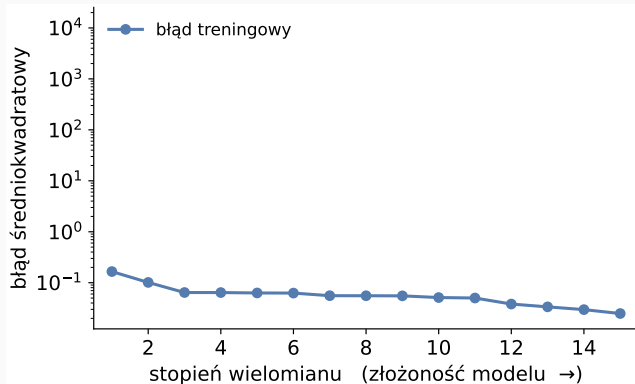
**W sam raz**: model odtwarza wzorec, ignoruje szum.



**Przeuczenie** (overfitting): model przechodzi przez każdy punkt, także przez szum. Świetny na treningu, fatalny na nowych danych.

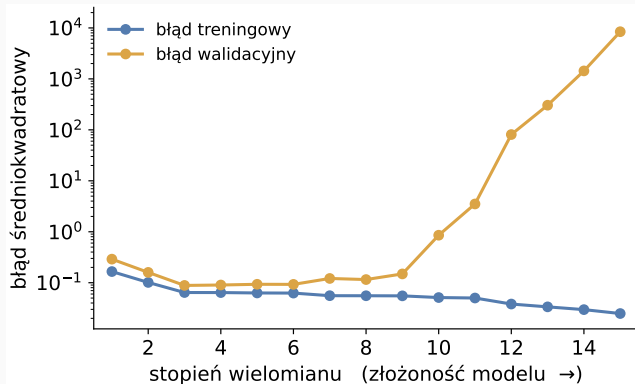
Zadanie modelu: wielomian  $\hat{y} = w_0 + w_1x + \dots + w_px^p$ . Stopień  $p$  steruje **złożonością** — to nasz „regulator”.

# Krzywa walidacyjna: błąd w funkcji złożoności



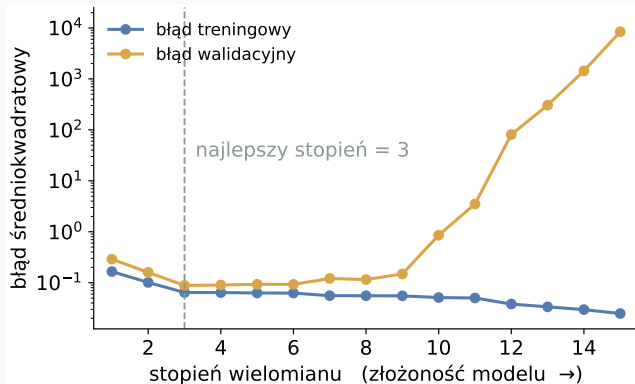
- Błąd **treningowy** maleje wraz ze złożonością — zawsze. Bardziej złożony model zawsze lepiej dopasuje się do tego, co widział.

# Krzywa walidacyjna: błąd w funkcji złożoności



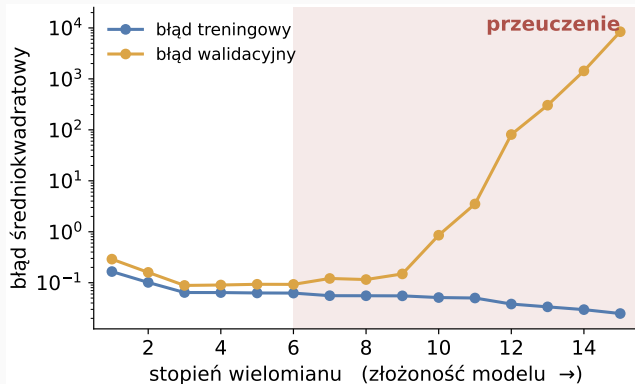
- Błąd **treningowy** maleje wraz ze złożonością — zawsze. Bardziej złożony model zawsze lepiej dopasuje się do tego, co widział.
- Błąd **walidacyjny** najpierw maleje, potem rośnie: kształt litery U.

# Krzywa walidacyjna: błąd w funkcji złożoności



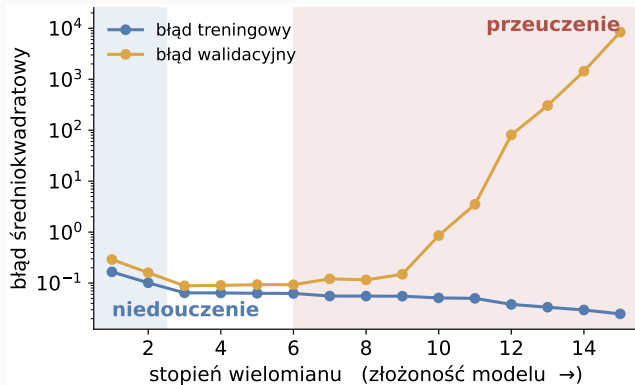
- Błąd **treningowy** maleje wraz ze złożonością — zawsze. Bardziej złożony model zawsze lepiej dopasuje się do tego, co widział.
- Błąd **walidacyjny** najpierw maleje, potem rośnie: kształt litery U.
- Szukamy dna litery U, *nie* minimum błędu treningowego.

# Krzywa walidacyjna: błąd w funkcji złożoności



**Duża przerwa** między krzywymi: model dopasował się do szumu w treningu. To **przeuczenie**.

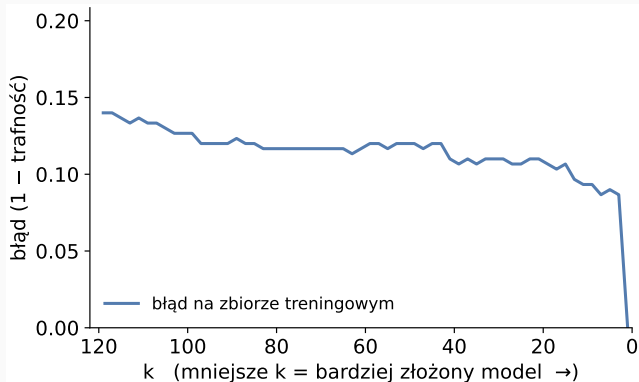
# Krzywa walidacyjna: błąd w funkcji złożoności



**Duża przerwa** między krzywymi: model dopasował się do szumu w treningu. To **przeuczenie**.

**Obie wysoko** i blisko siebie: model nie umie odwzorować nawet treningu. To **niedouczenie**.

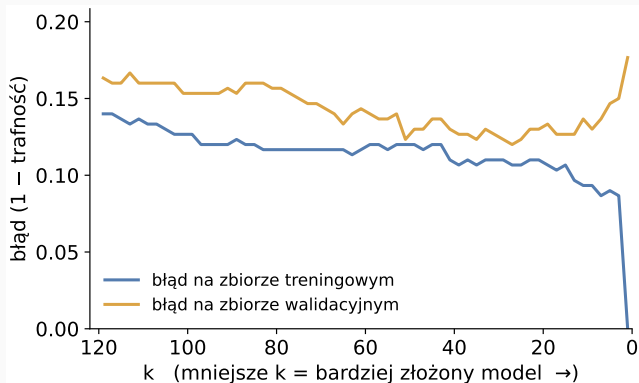
## To samo w kNN: $k$ jako regulator złożoności



- $k = 1$ : błąd treningowy = 0, bo każdy punkt jest swoim najbliższym sąsiadem. Im większe  $k$ , tym gorzej model odtwarza trening.

Uwaga: oś jest odwrócona — małe  $k$  to złożony model. Złożoność to „ile szczegółów model może odwzorować”, nie liczba parametrów.

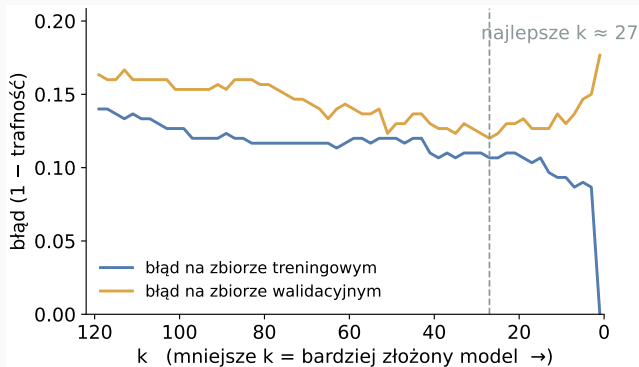
## To samo w kNN: $k$ jako regulator złożoności



- $k = 1$ : błąd treningowy = 0, bo każdy punkt jest swoim najbliższym sąsiadem. Im większe  $k$ , tym gorzej model odtwarza trening.
- Błąd walidacyjny: najpierw spada, gdy granica się wygładza, potem rośnie, gdy model ignoruje prawdziwą strukturę.

Uwaga: oś jest odwrócona — małe  $k$  to złożony model. Złożoność to „ile szczegółów model może odwzorować”, nie liczba parametrów.

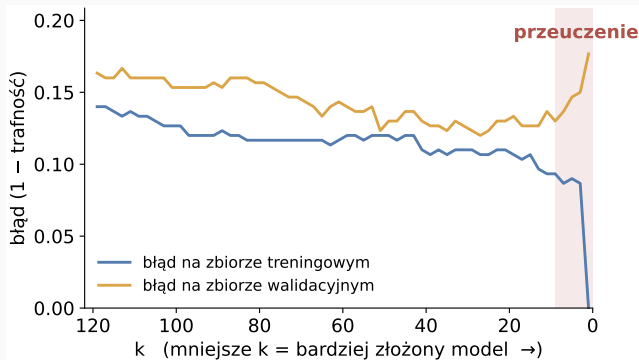
## To samo w kNN: $k$ jako regulator złożoności



- $k = 1$ : błąd treningowy = 0, bo każdy punkt jest swoim najbliższym sąsiadem. Im większe  $k$ , tym gorzej model odtwarza trening.
- Błąd walidacyjny: najpierw spada, gdy granica się wygładza, potem rośnie, gdy model ignoruje prawdziwą strukturę.
- Wybieramy  $k$  z dna litery U — tak samo jak stopień wielomianu.

Uwaga: oś jest odwrócona — małe  $k$  to złożony model. Złożoność to „ile szczegółów model może odwzorować”, nie liczba parametrów.

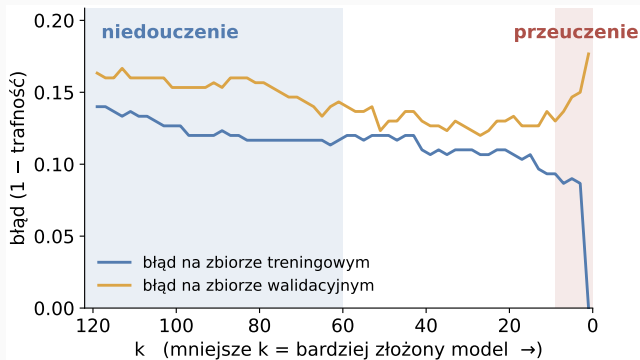
## To samo w kNN: $k$ jako regulator złożoności



**Małe  $k$ , duża przerwa** między krzywymi: model zapamiętał szum.  
To **przeuczenie**.

Uwaga: oś jest odwrócona — małe  $k$  to *złożony* model. Złożoność to „ile szczegółów model może odwzorować”, nie liczba parametrów.

## To samo w kNN: $k$ jako regulator złożoności



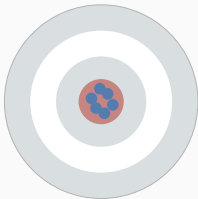
**Małe  $k$ , duża przerwa** między krzywymi: model zapamiętał szum. To **przeuczenie**.

**Duże  $k$ , obie wysoko**: model uśrednia po zbyt wielu sąsiadach i gubi kształt klas. To **niedouczenie**.

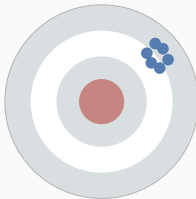
Uwaga: oś jest odwrócona — małe  $k$  to *złożony* model. Złożoność to „ile szczegółów model może odwzorować”, nie liczba parametrów.

# Obciążenie i wariancja: rzut do tarczy

małe obciążenie, mała wariancja



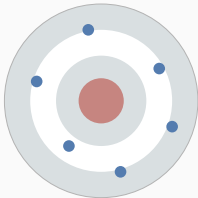
duże obciążenie, mała wariancja



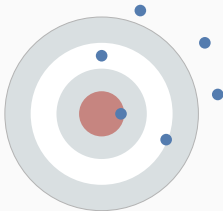
**Obciążenie (bias):**  
jak daleko *średni* strzał  
jest od środka.

**Wariancja:**  
jak bardzo strzały  
różnią się między sobą.

małe obciążenie, duża wariancja

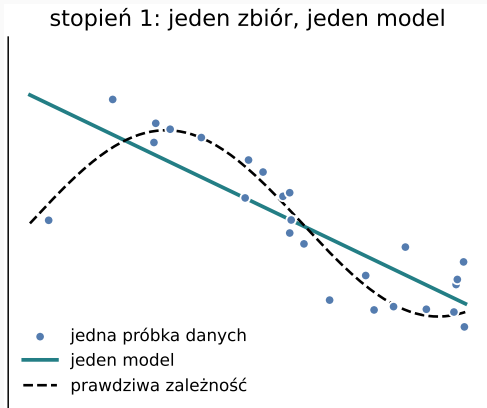


duże obciążenie, duża wariancja



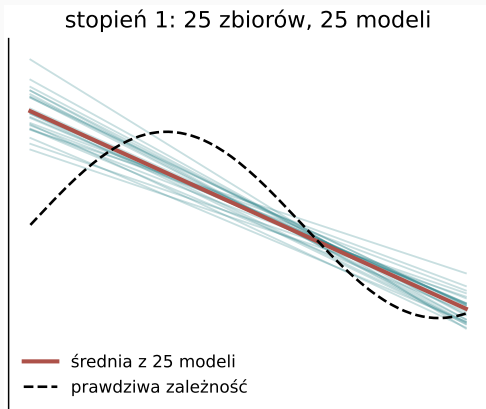
Jeden „strzał” = model  
nauczony na jednej  
losowej próbie danych.

# Ten sam eksperyment na wielomianach



Jeden „strzał”: losujemy 25 punktów i dopasowujemy prostą. Prosta nie umie być sinusoidą.

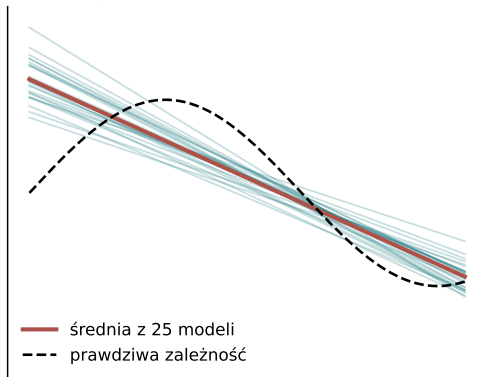
# Ten sam eksperyment na wielomianach



**Prosty model:** 25 losowań, 25 prostych —  
prawie takich samych (**mała wariancja**), ale  
ich średnia daleko od prawdy (**duże  
obciążenie**).

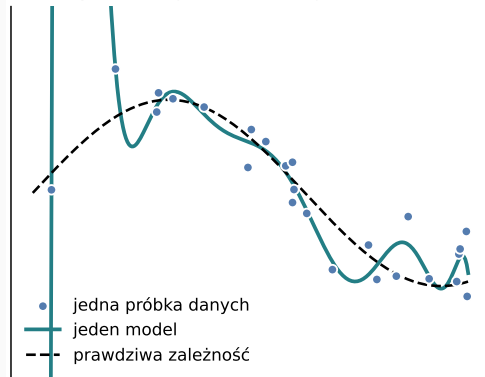
# Ten sam eksperyment na wielomianach

stopień 1: 25 zbiorów, 25 modeli



**Prosty model:** 25 losowań, 25 prostych — prawie takich samych (**mała wariancja**), ale ich średnia daleko od prawdy (**duże obciążenie**).

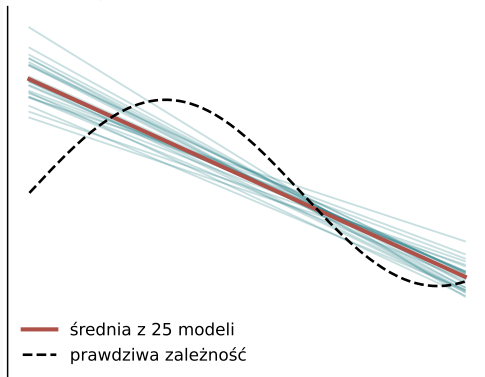
stopień 12: jeden zbiór, jeden model



Ten sam „strzał” wielomianem stopnia 12: przechodzi blisko punktów, także tam, gdzie to tylko szum.

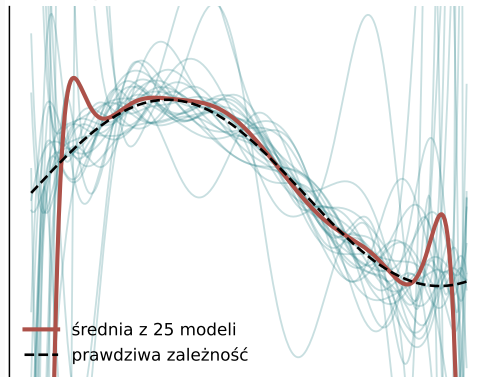
# Ten sam eksperyment na wielomianach

stopień 1: 25 zbiorów, 25 modeli



**Prosty model:** 25 losowań, 25 prostych — prawie takich samych (**mała wariancja**), ale ich średnia daleko od prawdy (**duże obciążenie**).

stopień 12: 25 zbiorów, 25 modeli

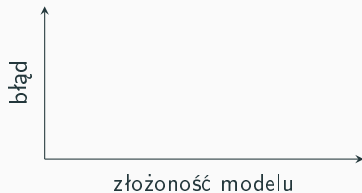


**Złożony model:** średnia z 25 modeli trafia w prawdziwą krzywą (**małe obciążenie**), ale każdy pojedynczy model wygląda inaczej (**duża wariancja**) — a my zawsze mamy tylko jeden.

# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

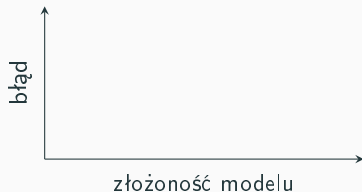
$$\mathbb{E}[(y - \hat{f}(x))^2]$$



# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

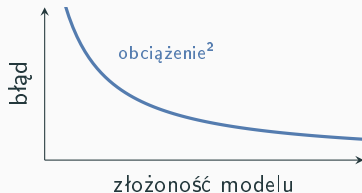
$$\mathbb{E}[(y - \hat{f}(x))^2] = (\mathbb{E}[\hat{f}(x)] - f(x))^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma_\varepsilon^2$$



# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{obciążenie}^2} + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma_\varepsilon^2$$

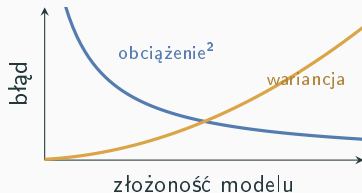


- **Obciążenie:** błąd z założeń modelu (prosta nie zegnije się w sinusoidę). Maleje ze złożonością.

# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{obciążenie}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{wariancja}} + \sigma_\varepsilon^2$$

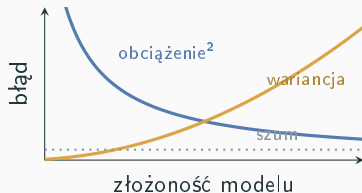


- **Obciążenie:** błąd z założeń modelu (prosta nie zegnije się w sinusoidę). Maleje ze złożonością.
- **Wariancja:** wrażliwość na to, które akurat dane wylosowaliśmy. Rośnie ze złożonością.

# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{obciążenie}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{wariancja}} + \underbrace{\sigma_\varepsilon^2}_{\text{szum}}$$

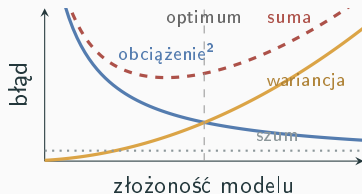


- **Obciążenie:** błąd z założeń modelu (prosta nie zegnije się w sinusoidę). Maleje ze złożonością.
- **Wariancja:** wrażliwość na to, które akurat dane wylosowaliśmy. Rośnie ze złożonością.
- **Szum:** nie do usunięcia żadnym modelem.

# Kompromis obciążenie–wariancja wzorem

Dla punktu  $x$ , prawdziwej zależności  $y = f(x) + \varepsilon$  i modelu  $\hat{f}$  nauczonego na losowym zbiorze:

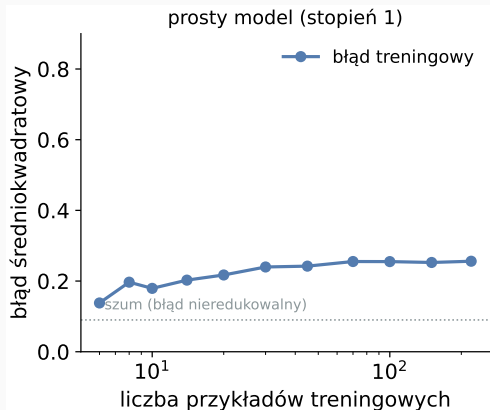
$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{obciążenie}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{wariancja}} + \underbrace{\sigma_\varepsilon^2}_{\text{szum}}$$



- **Obciążenie:** błąd z założeń modelu (prosta nie zegnije się w sinusoidę). Maleje ze złożonością.
- **Wariancja:** wrażliwość na to, które akurat dane wylosowaliśmy. Rośnie ze złożonością.
- **Szum:** nie do usunięcia żadnym modelem.

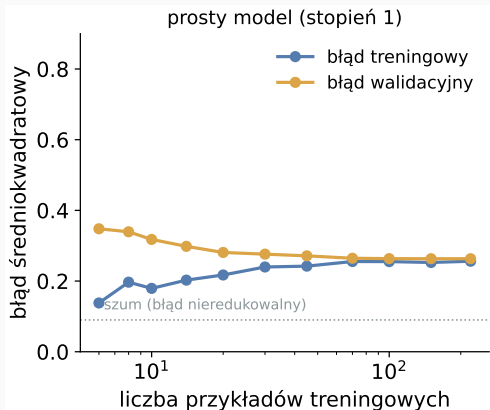
Zmniejszając jedno, zwykle zwiększamy drugie — stąd „kompromis”. Zespoły modeli (za chwilę) obniżają wariancję prawie „za darmo”.

# Krzywe uczenia: czy pomogą nowe dane?



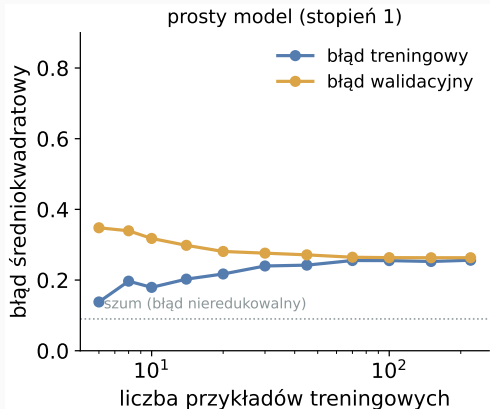
Oś pozioma: ile przykładów dostał model. Błąd treningowy *rośnie* z danymi: coraz trudniej dopasować się do wszystkich naraz.

# Krzywe uczenia: czy pomogą nowe dane?

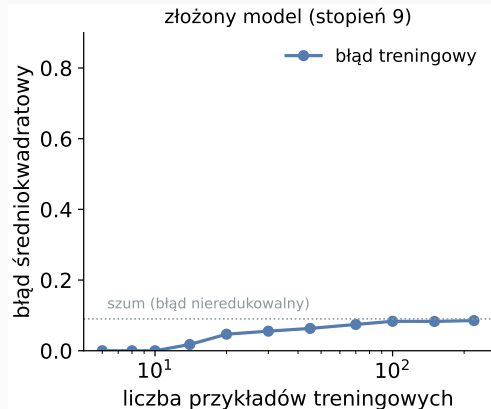


**Prosty model:** obie krzywe szybko zbiegają się wysoko. Więcej danych nie pomoże — trzeba bardziej złożonego modelu lub lepszych cech.

# Krzywe uczenia: czy pomogą nowe dane?

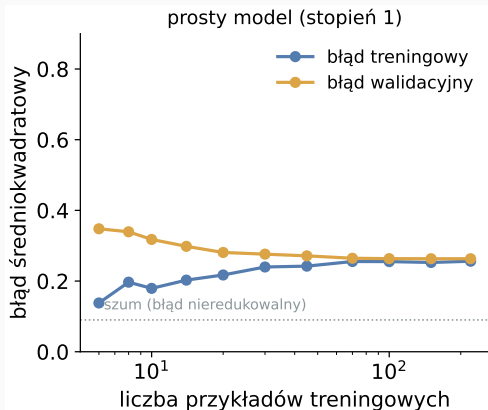


**Prosty model:** obie krzywe szybko zbiegają się wysoko. Więcej danych nie pomoże — trzeba bardziej złożonego modelu lub lepszych cech.

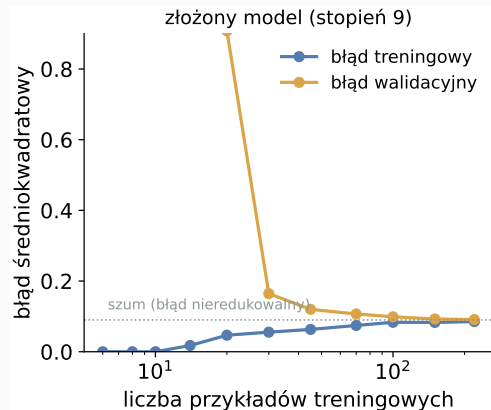


Złożony model przy kilku punktach ma błąd treningowy blisko zera: zapamiętał wszystko.

# Krzywe uczenia: czy pomogą nowe dane?



**Prosty model:** obie krzywe szybko zbiegają się *wysoko*. Więcej danych nie pomoże — trzeba bardziej złożonego modelu lub lepszych cech.



**Złożony model:** duża przerwa przy małej liczbie danych, która zamyka się w miarę ich dodawania. Tu więcej danych *pomaga*.

# Co robić, gdy model jest przeuczony

Mały błąd na treningu, duży na walidacji

**Zbierz więcej danych**



# Co robić, gdy model jest przeuczony

Mały błąd na treningu, duży na walidacji

**Zbierz więcej danych**



**Uprość model**



# Co robić, gdy model jest przeuczony

Mały błąd na treningu, duży na walidacji

**Zbierz więcej danych**



**Uprość model**



**Zwiększ regularyzację**



# Co robić, gdy model jest przeuczony

Mały błąd na treningu, duży na walidacji

**Zbierz więcej danych**



**Uprość model**



**Zwiększ regularyzację**



**Połącz modele**



# Co robić, gdy model jest niedouczony

Duży błąd na treningu i walidacji

**Zwiększ złożoność**



# Co robić, gdy model jest niedouczony

Duży błąd na treningu i walidacji

Zwiększ złożoność



Dodaj lepsze cechy



# Co robić, gdy model jest niedouczony

Duży błąd na treningu i walidacji

**Zwiększ złożoność**



**Dodaj lepsze cechy**



**Zmniejsz regularyzację**



# Co robić, gdy model jest niedouczony

Duży błąd na treningu i walidacji

**Zwiększ złożoność**



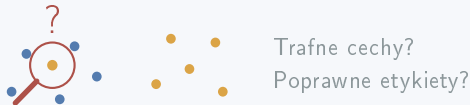
**Dodaj lepsze cechy**



**Zmniejsz regularyzację**



**Sprawdź dane**

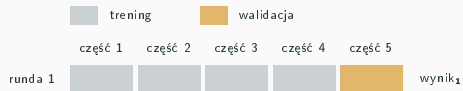


# Walidacja krzyżowa

---

# Jeden podział to jeden rzut kością

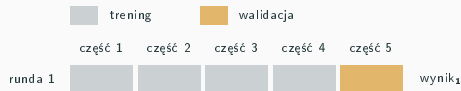
Przy 200 przykładach zbiór walidacyjny ma 40 punktów.



# Jeden podział to jeden rzut kością

Przy 200 przykładach zbiór walidacyjny ma 40 punktów.

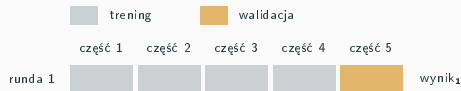
- Trafność 92,5 % vs 95 % to różnica **jednego** przykładu.



# Jeden podział to jeden rzut kością

Przy 200 przykładach zbiór walidacyjny ma 40 punktów.

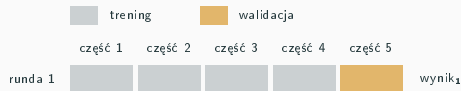
- Trafność 92,5 % vs 95 % to różnica **jednego** przykładu.
- Do zbioru walidacyjnego mogły trafić same „łatwe” punkty — albo same trudne.



# Jeden podział to jeden rzut kością

Przy 200 przykładach zbiór walidacyjny ma 40 punktów.

- Trafność 92,5 % vs 95 % to różnica **jednego** przykładu.
- Do zbioru walidacyjnego mogły trafić same „łatwe” punkty — albo same trudne.
- 20 % danych leży odłogiem podczas uczenia.



# Jeden podział to jeden rzut kością

Przy 200 przykładach zbiór walidacyjny ma 40 punktów.

- Trafność 92,5 % vs 95 % to różnica **jednego** przykładu.
- Do zbioru walidacyjnego mogły trafić same „łatwe” punkty — albo same trudne.
- 20 % danych leży odłogiem podczas uczenia.

**Pomysł:** podzielmy dane na  $K$  części i niech *każda* po kolei będzie zbiorem walidacyjnym.

|         |  trening |                                                                                     |  walidacja |                                                                                     |                                                                                     |                    |
|---------|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------|
|         | część 1                                                                                     | część 2                                                                             | część 3                                                                                       | część 4                                                                             | część 5                                                                             |                    |
| runda 1 |          |  |            |  |  | wynik <sub>1</sub> |
| runda 2 |          |  |            |  |  | wynik <sub>2</sub> |
| runda 3 |          |  |            |  |  | wynik <sub>3</sub> |
| runda 4 |          |  |            |  |  | wynik <sub>4</sub> |
| runda 5 |          |  |            |  |  | wynik <sub>5</sub> |

## K-krotna walidacja krzyżowa

Wynik to średnia z  $K$  rund, a rozrzut wyników mówi, jak bardzo można mu ufać:

$$\text{wynik}_{\text{cv}} = \frac{1}{K} \sum_{i=1}^K \text{wynik}_i, \quad \text{odchylenie} = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{wynik}_i - \text{wynik}_{\text{cv}})^2}$$

**Zalety**

## K-krotna walidacja krzyżowa

Wynik to średnia z  $K$  rund, a rozrzut wyników mówi, jak bardzo można mu ufać:

$$\text{wynik}_{\text{cv}} = \frac{1}{K} \sum_{i=1}^K \text{wynik}_i, \quad \text{odchylenie} = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{wynik}_i - \text{wynik}_{\text{cv}})^2}$$

### Zalety

- każdy przykład dokładnie raz trafia do walidacji — żadne dane nie leżą odłogiem

# K-krotna walidacja krzyżowa

Wynik to średnia z  $K$  rund, a rozrzut wyników mówi, jak bardzo można mu ufać:

$$\text{wynik}_{\text{cv}} = \frac{1}{K} \sum_{i=1}^K \text{wynik}_i, \quad \text{odchylenie} = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{wynik}_i - \text{wynik}_{\text{cv}})^2}$$

## Zalety

- każdy przykład dokładnie raz trafia do walidacji — żadne dane nie leżą odłogiem
- wynik uśredniony z  $K$  podziałów mniej zależy od szczęścia w losowaniu

# K-krotna walidacja krzyżowa

Wynik to średnia z  $K$  rund, a rozrzut wyników mówi, jak bardzo można mu ufać:

$$\text{wynik}_{\text{CV}} = \frac{1}{K} \sum_{i=1}^K \text{wynik}_i, \quad \text{odchylenie} = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{wynik}_i - \text{wynik}_{\text{CV}})^2}$$

## Zalety

- każdy przykład dokładnie raz trafia do walidacji — żadne dane nie leżą odłogiem
- wynik uśredniony z  $K$  podziałów mniej zależy od szczęścia w losowaniu
- dostajemy też miarę niepewności:  $92 \% \pm 1 \%$  to co innego niż  $92 \% \pm 6 \%$

# K-krotna walidacja krzyżowa

Wynik to średnia z  $K$  rund, a rozrzut wyników mówi, jak bardzo można mu ufać:

$$\text{wynik}_{\text{CV}} = \frac{1}{K} \sum_{i=1}^K \text{wynik}_i, \quad \text{odchylenie} = \sqrt{\frac{1}{K} \sum_{i=1}^K (\text{wynik}_i - \text{wynik}_{\text{CV}})^2}$$

## Zalety

- każdy przykład dokładnie raz trafia do walidacji — żadne dane nie leżą odłogiem
- wynik uśredniony z  $K$  podziałów mniej zależy od szczęścia w losowaniu
- dostajemy też miarę niepewności:  $92\% \pm 1\%$  to co innego niż  $92\% \pm 6\%$

## Koszt

Trenujemy  $K$  modeli zamiast jednego. Dla  $K = 5$  i 32 kombinacji hiperparametrów to 160 treningów.

# Walidacja krzyżowa w kodzie

```
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(knn, X_train, y_train, cv=cv, scoring="f1_macro")

print(scores)                # 5 wyników, po jednym na rundę
print(scores.mean(), scores.std()) # średnia i odchylenie
```

- `cross_val_score` sam dzieli dane, trenuje  $K$  razy i zwraca  $K$  wyników.
- `shuffle=True`: przed podziałem tasujemy — dane bywają posortowane po etykiecie.
- `scoring`: ta sama metryka, którą będziemy raportować (o metrykach za chwilę).
- Uwaga: podajemy `X_train`, nie całe `X`. Test nadal leży odłożony.

$K = 5$  lub  $K = 10$  standard. Większe  $K$ : więcej danych na trening w każdej rundzie, ale więcej treningów i bardziej podobne rundy.

**Stratified  $K$ -fold** każda część ma te same proporcje klas co całość. Dla klasyfikacji prawie zawsze; bez tego rzadka klasa może w którejś rundzie nie trafić do walidacji.

**Leave-one-out**  $K = n$ : w każdej rundzie walidacją jest jeden przykład. Bez losowości i maksimum danych na trening, ale  $n$  treningów, a rundy są niemal identyczne, więc wyniki mocno skorelowane.

**Powtórzona CV** całe  $K$ -fold kilka razy z innym tasowaniem; uśredniamy jeszcze więcej podziałów.

# Warianty walidacji krzyżowej

$K = 5$  lub  $K = 10$  standard. Większe  $K$ : więcej danych na trening w każdej rundzie, ale więcej treningów i bardziej podobne rundy.

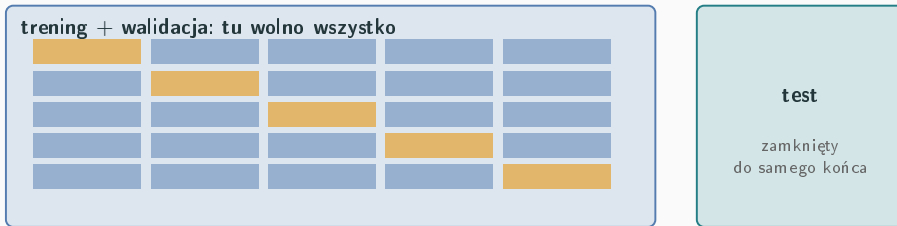
**Stratified  $K$ -fold** każda część ma te same proporcje klas co całość. Dla klasyfikacji prawie zawsze; bez tego rzadka klasa może w którejś rundzie nie trafić do walidacji.

**Leave-one-out**  $K = n$ : w każdej rundzie walidacją jest jeden przykład. Bez losowości i maksimum danych na trening, ale  $n$  treningów, a rundy są niemal identyczne, więc wyniki mocno skorelowane.

**Powtórzona CV** całe  $K$ -fold kilka razy z innym tasowaniem; uśredniamy jeszcze więcej podziałów.

W praktyce: stratified  $K$ -fold z  $K = 5$  lub 10 to dobry domyślny wybór.

# Gdzie w tym wszystkim jest zbiór testowy?



walidacja krzyżowa wybiera  $k$ , model i cechy; potem trenujemy wybrany model na całej lewej części  
i **raz** oceniamy go na prawej

**Metryki: po czym poznać dobry model?**

---

# Macierz pomyłek

|        |            | Predykcja modelu                   |                                   |
|--------|------------|------------------------------------|-----------------------------------|
|        |            | chory (+)                          | zdrowy (-)                        |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie<br>dodatni | <b>FN</b><br>fałszywie<br>ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie<br>dodatni  | <b>TN</b><br>prawdziwie<br>ujemny |

Umawiamy się, która klasa jest „dodatnia”: zwykle ta, której szukamy (chory, spam, oszustwo).

# Macierz pomyłek

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

Umawiamy się, która klasa jest „dodatnia”: zwykle ta, której szukamy (chory, spam, oszustwo).

- **TP, TN**: model ma rację.

# Macierz pomyłek

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

Umawiamy się, która klasa jest „dodatnia”: zwykle ta, której szukamy (chory, spam, oszustwo).

- **TP, TN**: model ma rację.
- **FP** (fałszywy alarm): zdrowy uznany za chorego.
- **FN** (przeoczenie): chory uznany za zdrowego.

# Macierz pomyłek

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

Umawiamy się, która klasa jest „dodatnia”: zwykle ta, której szukamy (chory, spam, oszustwo).

- **TP, TN**: model ma rację.
- **FP** (fałszywy alarm): zdrowy uznany za chorego.
- **FN** (przeoczenie): chory uznany za zdrowego.

## Filtr spamu, 100 wiadomości

|                   | spam?   | nie-spam? |
|-------------------|---------|-----------|
| naprawdę spam     | TP = 35 | FN = 5    |
| naprawdę nie-spam | FP = 8  | TN = 52   |

FP: ważny mail w spamie (irytujące). FN: spam w skrzynce (groźne, gdy to phishing).

$$\text{trafność (accuracy)} = \frac{TP + TN}{TP + TN + FP + FN} = \text{odsetek poprawnych odpowiedzi}$$

Dla filtru spamu:  $(35 + 52)/100 = 87\%$ . Brzmi dobrze i *jest* dobrze — klasy są w miarę zrównoważone.

# Trafność i jej pułapka

$$\text{trafność (accuracy)} = \frac{TP + TN}{TP + TN + FP + FN} = \text{odsetek poprawnych odpowiedzi}$$

Dla filtru spamu:  $(35 + 52)/100 = 87\%$ . Brzmi dobrze i *jest* dobrze — klasy są w miarę zrównoważone.

## Rzadka choroba: 1 % populacji

Model „wszyscy są zdrowi”:

$$\text{trafność} = \frac{0 + 990}{1000} = 99\%$$

i nie wykrył **ani jednego** chorego.

# Trafność i jej pułapka

$$\text{trafność (accuracy)} = \frac{TP + TN}{TP + TN + FP + FN} = \text{odsetek poprawnych odpowiedzi}$$

Dla filtru spamu:  $(35 + 52)/100 = 87\%$ . Brzmi dobrze i *jest* dobrze — klasy są w miarę zrównoważone.

## Rzadka choroba: 1 % populacji

Model „wszyscy są zdrowi”:

$$\text{trafność} = \frac{0 + 990}{1000} = 99\%$$

i nie wykrył **ani jednego** chorego.

Trafność jest myląca, gdy:

- klasy są **niezrównoważone** (jedna jest rzadka),
- błędy FP i FN mają **różny koszt**.

Zawsze porównuj z prostym punktem odniesienia: klasą większościową.

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

$$\text{precyzja} = \frac{TP}{TP + FP}$$

„Gdy model mówi *chory*, jak często ma rację?”

Ważna, gdy fałszywy alarm jest kosztowny:  
niepotrzebna operacja, ważny mail w spamie,  
niesłuszne oskarżenie.

# Precyzja, czułość, swoistość

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

$$\text{czułość (recall, TPR)} = \frac{TP}{TP + FN}$$

„Jaką część *naprawdę chorych* model wykrył?”

Ważna, gdy przeoczenie jest kosztowne: nowotwór, oszustwo kartą, wada w silniku samolotu.

# Precyzja, czułość, swoistość

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

$$\text{swoistość (specificity, TNR)} = \frac{TN}{TN + FP}$$

„Jaką część *naprawdę zdrowych* model poprawnie uspokoił?”

Lustrzane odbicie czułości dla klasy ujemnej;

$1 - \text{swoistość} = FPR$  (odsetek fałszywych alarmów).

|        |            | Predykcja modelu                |                                |
|--------|------------|---------------------------------|--------------------------------|
|        |            | chory (+)                       | zdrowy (-)                     |
| Prawda | chory (+)  | <b>TP</b><br>prawdziwie dodatni | <b>FN</b><br>fałszywie ujemny  |
|        | zdrowy (-) | <b>FP</b><br>fałszywie dodatni  | <b>TN</b><br>prawdziwie ujemny |

$$\text{swoistość (specificity, TNR)} = \frac{TN}{TN + FP}$$

„Jaką część *naprawdę zdrowych* model poprawnie uspokoił?”

Lustrzane odbicie czułości dla klasy ujemnej;

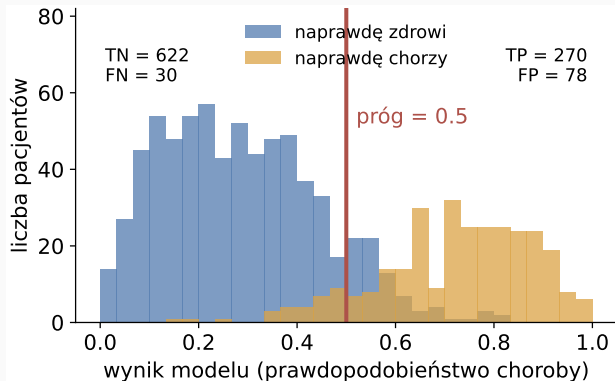
$1 - \text{swoistość} = FPR$  (odsetek fałszywych alarmów).

## Filtr spamu

precyzja =  $35/43 = 0,81$ ,    czułość =  $35/40 = 0,88$ ,

swoistość =  $52/60 = 0,87$

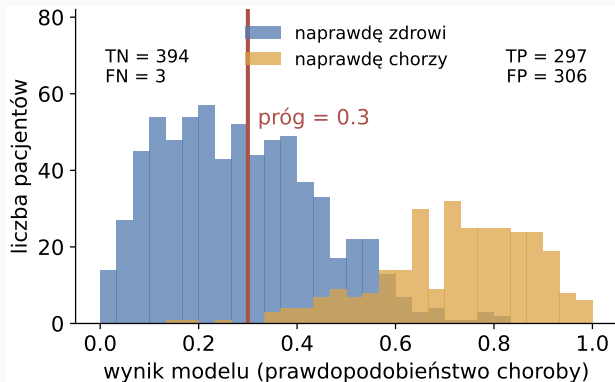
# Model nie mówi „chory”, mówi „0,73”



Większość klasyfikatorów zwraca **wynik** (np. prawdopodobieństwo). Decyzję podejmuje **próg**: „chory, jeśli wynik  $\geq t$ ”.

| próg $t$ | precyzja | czułość | swoistość |
|----------|----------|---------|-----------|
| 0,5      | 0,78     | 0,90    | 0,89      |

# Model nie mówi „chory”, mówi „0,73”

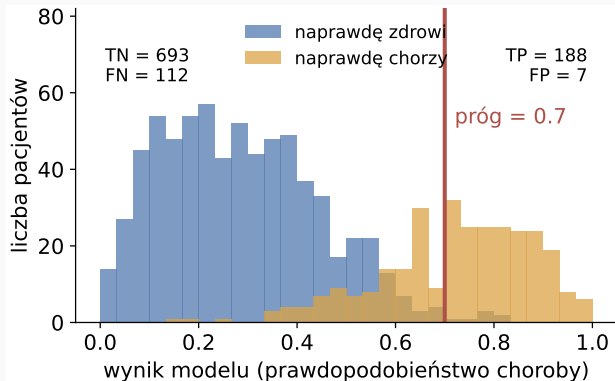


Większość klasyfikatorów zwraca **wynik** (np. prawdopodobieństwo). Decyzję podejmuje **próg**: „chory, jeśli wynik  $\geq t$ ”.

| próg $t$ | precyzja | czułość | swoistość |
|----------|----------|---------|-----------|
| 0,5      | 0,78     | 0,90    | 0,89      |
| 0,3      | 0,49     | 0,99    | 0,56      |

Niski próg: wykrywamy prawie wszystkich chorych, ale połowa alarmów jest fałszywa.

# Model nie mówi „chory”, mówi „0,73”

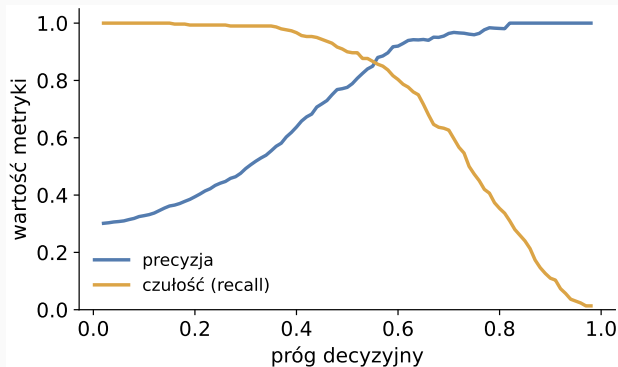


Większość klasyfikatorów zwraca **wynik** (np. prawdopodobieństwo). Decyzję podejmuje **próg**: „chory, jeśli wynik  $\geq t$ ”.

| próg $t$ | precyzja | czułość | swoistość |
|----------|----------|---------|-----------|
| 0,5      | 0,78     | 0,90    | 0,89      |
| 0,3      | 0,49     | 0,99    | 0,56      |
| 0,7      | 0,96     | 0,63    | 0,99      |

Niski próg: wykrywamy prawie wszystkich chorych, ale połowa alarmów jest fałszywa.  
Wysoki próg: alarmy niemal pewne, ale przeoczamy co trzeciego chorego.

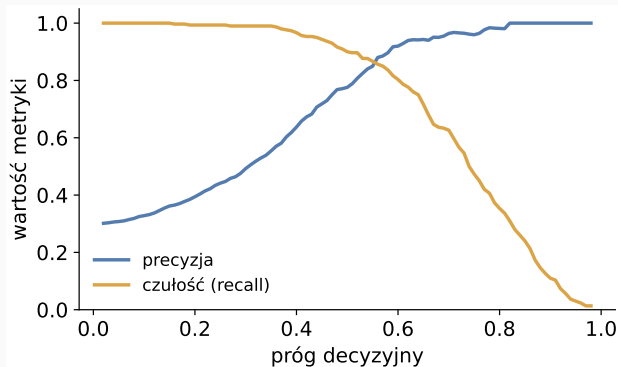
# Precyzja kontra czułość: nie da się mieć obu za darmo



Przesuwając próg, **wymieniamy** precyzję na czułość.

- badanie przesiewowe: wysoka czułość (lepiej sprawdzić zdrowego niż przeoczyć chorego)
- automatyczne blokowanie konta: wysoka precyzja (lepiej przepuścić oszusta niż zablokować uczciwego)

# Precyzja kontra czułość: nie da się mieć obu za darmo

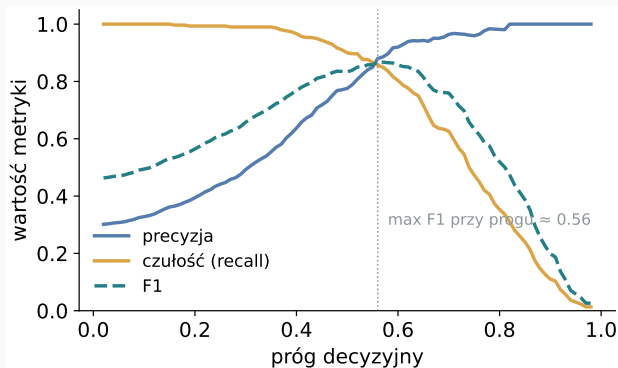


Przesuwając próg, **wymieniamy** precyzję na czułość.

- badanie przesiewowe: wysoka czułość (lepiej sprawdzić zdrowego niż przeoczyć chorego)
- automatyczne blokowanie konta: wysoka precyzja (lepiej przepuścić oszusta niż zablokować uczciwego)

Próg dobieramy **na zbiorze walidacyjnym** pod konkretny koszt błędów — to też hiperparametr.

# Precyzja kontra czułość: nie da się mieć obu za darmo



Przesuwając próg, **wymieniamy** precyzję na czułość.

- badanie przesiewowe: wysoka czułość (lepiej sprawdzić zdrowego niż przeoczyć chorego)
- automatyczne blokowanie konta: wysoka precyzja (lepiej przepuścić oszusta niż zablokować uczciwego)

Próg dobieramy **na zbiorze walidacyjnym** pod konkretny koszt błędów — to też hiperparametr.

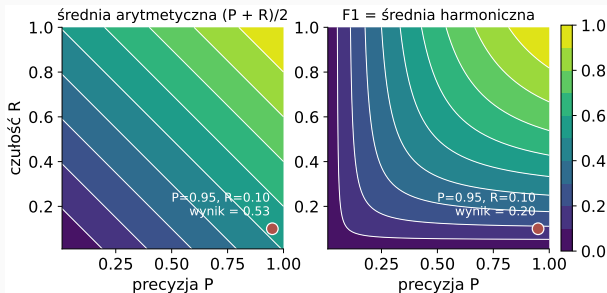
A gdy nie wiemy, co jest ważniejsze? Potrzebna jedna liczba, która łączy obie:  $F_1$  (następny slajd).

## Jedna liczba zamiast dwóch: $F_1$ i $F_\beta$

$$F_1 = \frac{2 \cdot \text{precyzja} \cdot \text{czułość}}{\text{precyzja} + \text{czułość}} = \frac{2 TP}{2 TP + FP + FN}$$

# Jedna liczba zamiast dwóch: $F_1$ i $F_\beta$

$$F_1 = \frac{2 \cdot \text{precyzja} \cdot \text{czułość}}{\text{precyzja} + \text{czułość}} = \frac{2 TP}{2 TP + FP + FN}$$



## Dlaczego średnia harmoniczna?

Precyzja 0,95, czułość 0,10:

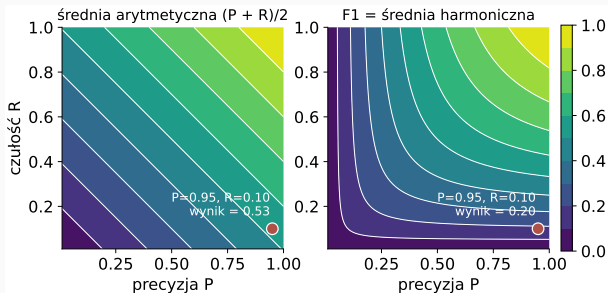
- średnia arytmetyczna: 0,53 — „nieźle”
- $F_1$ : 0,18 — „źle”

Harmoniczna **karze** za to, że jedna z wartości jest niska: żeby  $F_1$  było wysokie, *obie* muszą być wysokie.

# Jedna liczba zamiast dwóch: $F_1$ i $F_\beta$

$$F_1 = \frac{2 \cdot \text{precyzja} \cdot \text{czułość}}{\text{precyzja} + \text{czułość}} = \frac{2 TP}{2 TP + FP + FN}$$

$$F_\beta = (1 + \beta^2) \frac{\text{precyzja} \cdot \text{czułość}}{\beta^2 \cdot \text{precyzja} + \text{czułość}}$$



## Dlaczego średnia harmoniczna?

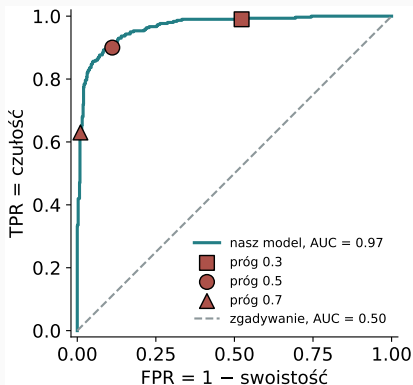
Precyzja 0,95, czułość 0,10:

- średnia arytmetyczna: 0,53 — „nieźle”
- $F_1$ : 0,18 — „źle”

Harmoniczna **karze** za to, że jedna z wartości jest niska: żeby  $F_1$  było wysokie, *obie* muszą być wysokie.

$F_\beta$ :  $\beta$  mówi, ile razy czułość jest ważniejsza od precyzji.  $\beta = 2$  premiuje czułość ( $F_2$  w medycynie),

# Krzywa ROC: wszystkie progi naraz

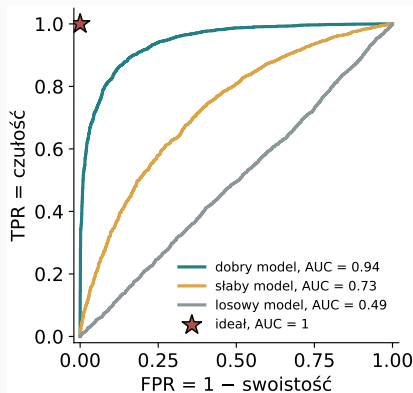


Dla każdego progu  $t$  rysujemy punkt

$$\left( \underbrace{FPR(t)}_{1-\text{swoistość}}, \underbrace{TPR(t)}_{\text{czułość}} \right).$$

Nasze trzy progi to trzy punkty jednej krzywej. Próg 0,3 (kwadrat): czułość 0,99, ale FPR 0,44. Próg 0,7 (trójkąt): FPR 0,01, czułość 0,63.

# Krzywa ROC: wszystkie progi naraz



Dla każdego progu  $t$  rysujemy punkt

$$\left( \underbrace{FPR(t)}_{1 - \text{swoistość}}, \underbrace{TPR(t)}_{\text{czułość}} \right).$$

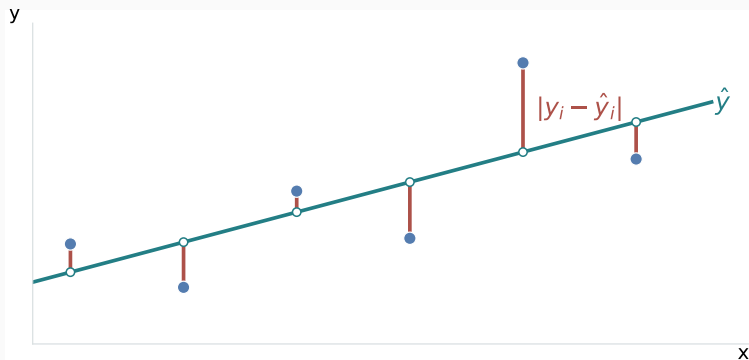
**AUC** — pole pod krzywą: 1,0 to model idealny (lewy górny róg), 0,5 to zgadywanie (przekątna). Nie zależy od progu ani od proporcji klas.

## Interpretacja AUC

Losujemy jednego chorego i jednego zdrowego. AUC = prawdopodobieństwo, że model da choremu wyższy wynik.

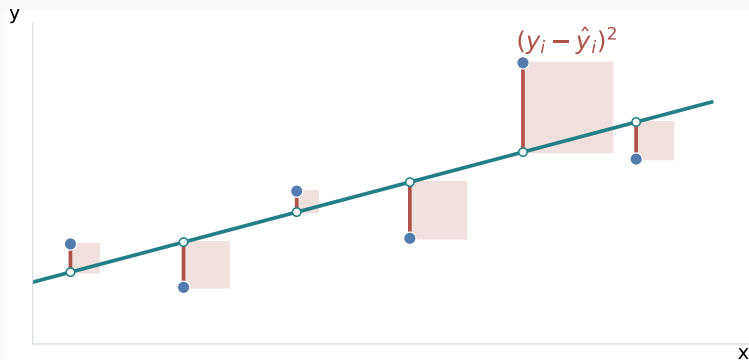
# Metryki regresji: MAE

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



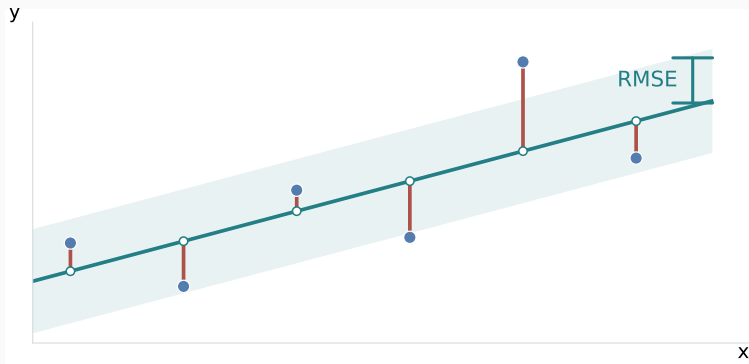
# Metryki regresji: MSE

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



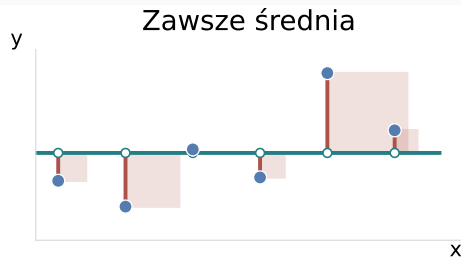
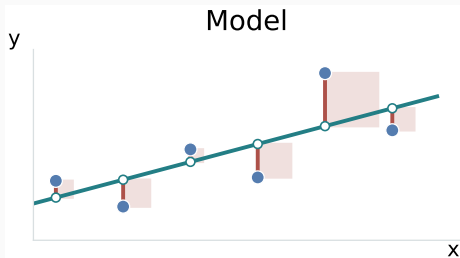
# Metryki regresji: RMSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} = \sqrt{\text{MSE}}$$



# Metryki regresji: $R^2$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$



## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |           |          |       |
|--------|--------------|-----------|----------|-------|
| prawda | kot          | pies      | królik   | razem |
| kot    | <b>40</b>    | 5         | 5        | 50    |
| pies   | 10           | <b>30</b> | 0        | 40    |
| królik | 0            | 2         | <b>8</b> | 10    |
| razem  | 50           | 37        | 13       | 100   |

100 zwierząt, 3 klasy. Wiersz: co było naprawdę.

Kolumna: co powiedział model. Przekątna:  
trafienia.

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |      |        |       |         |
|--------|--------------|------|--------|-------|---------|
| prawda | kot          | pies | królik | razem | czułość |
| kot    | 40           | 5    | 5      | 50    | 0,80    |
| pies   | 10           | 30   | 0      | 40    | 0,75    |
| królik | 0            | 2    | 8      | 10    | 0,80    |
| razem  | 50           | 37   | 13     | 100   |         |

**Czułość klasy:** ile prawdziwych kotów model rozpoznał jako koty.

$$\text{czułość}_{\text{kot}} = \frac{40}{50} = 0,80$$

$$\text{czułość}_{\text{pies}} = \frac{30}{40} = 0,75$$

Licznik: przekątna. Mianownik: suma *wiersza*.

Każda klasa po kolei gra „dodatnią” przeciw pozostałym (one-vs-rest).

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |      |        |       |         |          |
|--------|--------------|------|--------|-------|---------|----------|
| prawda | kot          | pies | królik | razem | czułość | precyzja |
| kot    | 40           | 5    | 5      | 50    | 0,80    | 0,80     |
| pies   | 10           | 30   | 0      | 40    | 0,75    | 0,81     |
| królik | 0            | 2    | 8      | 10    | 0,80    | 0,62     |
| razem  | 50           | 37   | 13     | 100   |         |          |

**Precyzja klasy:** gdy model mówi „królik”, jak często to królik.

$$\text{precyzja}_{\text{królik}} = \frac{8}{13} = 0,62$$

$$\text{precyzja}_{\text{pies}} = \frac{30}{37} = 0,81$$

Licznik: przekątna. Mianownik: suma *kolumny*.

Każda klasa po kolei gra „dodatnią” przeciw pozostałym (one-vs-rest).

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |      |        |       |         |          |
|--------|--------------|------|--------|-------|---------|----------|
| prawda | kot          | pies | królik | razem | czułość | precyzja |
| kot    | 40           | 5    | 5      | 50    | 0,80    | 0,80     |
| pies   | 10           | 30   | 0      | 40    | 0,75    | 0,81     |
| królik | 0            | 2    | 8      | 10    | 0,80    | 0,62     |
| razem  | 50           | 37   | 13     | 100   |         |          |

**Macro:** zwykła średnia po klasach.

$$\text{czułość}_{\text{macro}} = \frac{0,80 + 0,75 + 0,80}{3} = 0,78$$

Każda klasa waży tyle samo: królik (10 sztuk) liczy się tak samo jak kot (50).

Trzy liczby na klasę trzeba jakoś **uśrednić** do jednej. Są trzy sposoby.

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |      |        |       |         |          |
|--------|--------------|------|--------|-------|---------|----------|
| prawda | kot          | pies | królik | razem | czułość | precyzja |
| kot    | 40           | 5    | 5      | 50    | 0,80    | 0,80     |
| pies   | 10           | 30   | 0      | 40    | 0,75    | 0,81     |
| królik | 0            | 2    | 8      | 10    | 0,80    | 0,62     |
| razem  | 50           | 37   | 13     | 100   |         |          |

**Weighted:** średnia ważona liczebnością klas.

$$\text{precyzja}_w = \frac{50}{100} \cdot 0,80 + \frac{40}{100} \cdot 0,81 + \frac{10}{100} \cdot 0,62 = 0,79$$

Wagi to sumy wierszy. Duże klasy dominują; słaba rzadka klasa ginie w średniej.

Trzy liczby na klasę trzeba jakoś **uśrednić** do jednej. Są trzy sposoby.

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |      |        |       |         |          |
|--------|--------------|------|--------|-------|---------|----------|
| prawda | kot          | pies | królik | razem | czułość | precyzja |
| kot    | 40           | 5    | 5      | 50    | 0,80    | 0,80     |
| pies   | 10           | 30   | 0      | 40    | 0,75    | 0,81     |
| królik | 0            | 2    | 8      | 10    | 0,80    | 0,62     |
| razem  | 50           | 37   | 13     | 100   |         |          |

**Micro:** najpierw sumujemy TP, FP, FN ze wszystkich klas, dopiero potem liczymy metrykę.

$$\text{precyzja}_{\text{micro}} = \frac{40 + 30 + 8}{100} = 0,78$$

Dla jednej etykiety na przykład  
micro-precyzja = micro-czułość = **trafność**.

Trzy liczby na klasę trzeba jakoś **uśrednić** do jednej. Są trzy sposoby.

## Więcej niż dwie klasy: macro, micro, weighted

|        | przewidziano |           |          |       |         |          |
|--------|--------------|-----------|----------|-------|---------|----------|
| prawda | kot          | pies      | królik   | razem | czułość | precyzja |
| kot    | <b>40</b>    | 5         | 5        | 50    | 0,80    | 0,80     |
| pies   | 10           | <b>30</b> | 0        | 40    | 0,75    | 0,81     |
| królik | 0            | 2         | <b>8</b> | 10    | 0,80    | 0,62     |
| razem  | 50           | 37        | 13       | 100   |         |          |

### Którą wybrać?

- rzadka klasa jest ważna (królik)  $\Rightarrow$  **macro**
- liczy się ogólny odsetek trafień  $\Rightarrow$  micro (czyli trafność)
- weighted: kompromis, ale ukrywa słabe rzadkie klasy

W `classification_report` ze `scikit-learn` dostajemy wszystkie trzy naraz.

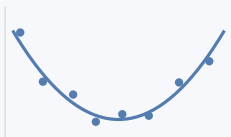
# Dobór hiperparametrów

---

# Parametry a hiperparametry

## Parametry

Model uczy się ich z danych.



$w_0, w_1, w_2$

## Hiperparametry

Ustawiamy je przed treningiem.

Stopień wielomianu

1

2

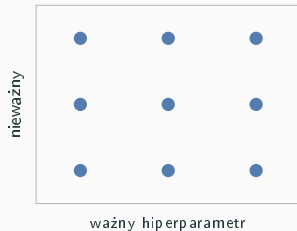
3

$p = 2$

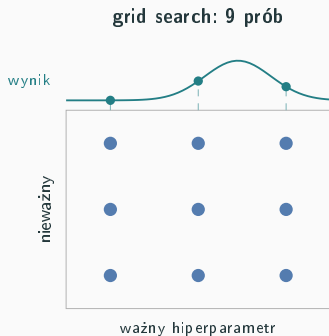
Hiperparametry dobieramy na walidacji.

# Grid search i random search

grid search: 9 prób

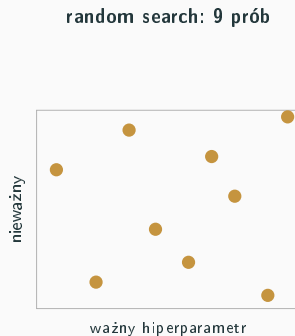
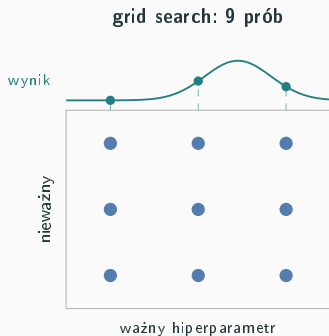


# Grid search i random search



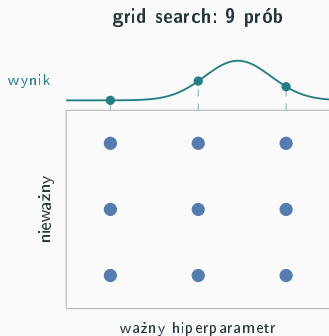
**Grid search:** sprawdź każdą kombinację z siatki. 9 prób, ale tylko **3 różne** wartości ważnego parametru — 6 prób powtarza to, co już wiemy. Koszt: iloczyn rozmiarów siatek  $\times$  liczba części CV.

# Grid search i random search

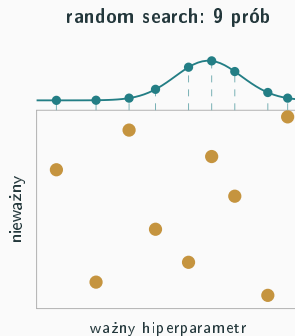


**Grid search:** sprawdź każdą kombinację z siatki. 9 prób, ale tylko **3 różne** wartości ważnego parametru — 6 prób powtarza to, co już wiemy. Koszt: iloczyn rozmiarów siatek  $\times$  liczba części CV.

# Grid search i random search



**Grid search:** sprawdź każdą kombinację z siatki. 9 prób, ale tylko **3 różne** wartości ważnego parametru — 6 prób powtarza to, co już wiemy. Koszt: iloczyn rozmiarów siatek  $\times$  liczba części CV.



**Random search:** losuj kombinacje. Te same 9 prób daje **9 różnych** wartości ważnego parametru, więc łatwiej trafić blisko szczytu. Zwykle wygrywa, gdy hiperparametrów jest więcej niż 2–3.

# Grid search w kodzie

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

param_grid = {"n_neighbors": [1, 3, 5, 7, 11, 15, 21, 31],
              "weights": ["uniform", "distance"],
              "p": [1, 2]}                                # 8 * 2 * 2 = 32 kombinacje

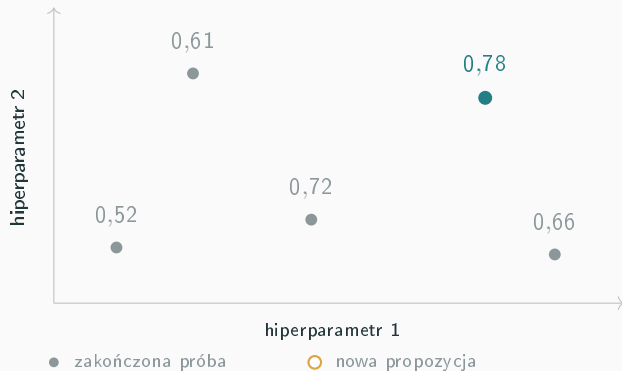
search = GridSearchCV(KNeighborsClassifier(), param_grid,
                      cv=5, scoring="f1_macro", n_jobs=-1) # 32 * 5 = 160 treningów
search.fit(X_train, y_train)

print(search.best_params_, search.best_score_)
model = search.best_estimator_                        # gotowy, wytrenowany na całym X_train
print(model.score(X_test, y_test))                   # teraz -- i tylko teraz -- test
```

RandomizedSearchCV(..., n\_iter=40) ma ten sam interfejs: zamiast list podajemy rozkłady (np. randint(1, 50)) i liczbę prób.

# Optuna: uczyć się z poprzednich prób

Wynik na walidacji: więcej = lepiej



KROK 1 / 5

## Zacznij losowo

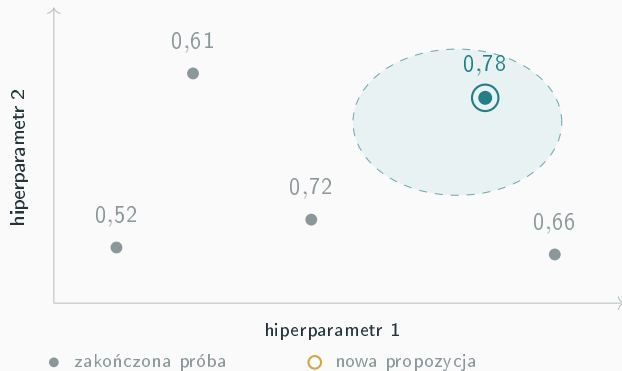
Każdy punkt to inny zestaw hiperparametrów.

Najlepszy wynik

0,78

# Optuna: uczyć się z poprzednich prób

Wynik na walidacji: więcej = lepiej



KROK 2 / 5

## Znajdź obszar

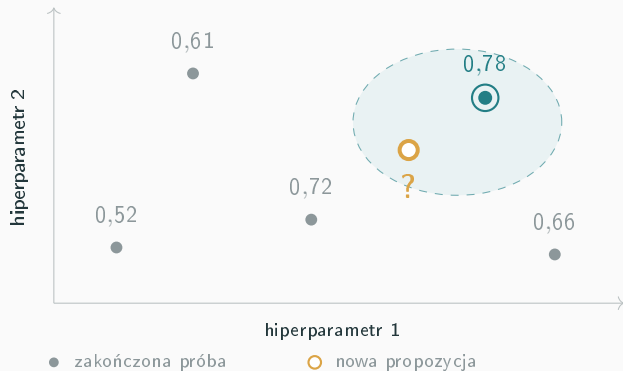
Dotychczasowe wyniki podpowiadają, gdzie szukać dalej.

Najlepszy wynik

0,78

# Optuna: uczyć się z poprzednich prób

Wynik na walidacji: więcej = lepiej



KROK 3 / 5

## Sprawdź propozycję

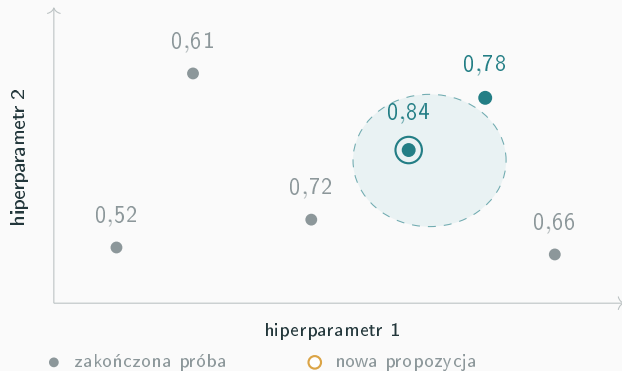
Wytrenuj model i zmierz  
wynik nowej próby.

Najlepszy wynik

0,78

# Optuna: uczyć się z poprzednich prób

Wynik na walidacji: więcej = lepiej



KROK 4 / 5

## Dopisz wynik

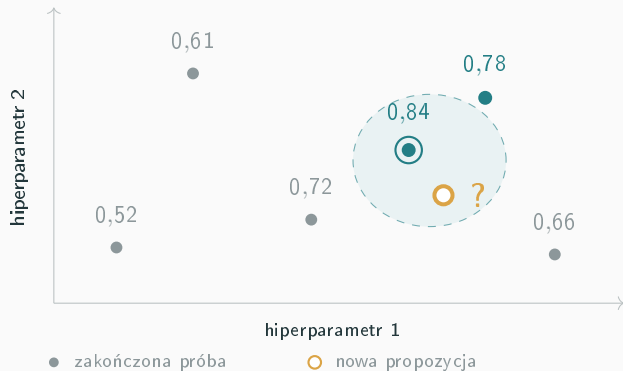
Nowy wynik zmienia wybór kolejnych propozycji.

Najlepszy wynik

0,84

# Optuna: uczyć się z poprzednich prób

Wynik na walidacji: więcej = lepiej



KROK 5 / 5

## Wybierz ponownie

Powtarzaj, aż wykorzystasz budżet prób.

Najlepszy wynik

0,84

# Optuna w kodzie

```
import optuna

def objective(trial):
    k = trial.suggest_int("n_neighbors", 1, 50, log=True)
    w = trial.suggest_categorical("weights", ["uniform", "distance"])
    model = KNeighborsClassifier(n_neighbors=k, weights=w)
    return cross_val_score(model, X_train, y_train, cv=5, scoring="f1_macro").mean()

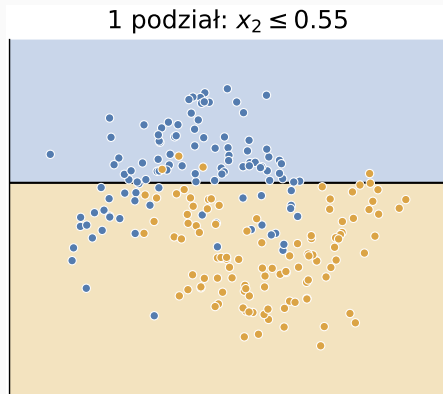
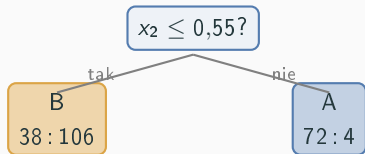
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=60)
print(study.best_params, study.best_value)
```

- `objective` to zwykła funkcja: bierze propozycję hiperparametrów, zwraca wynik walidacyjny.
- Przestrzeń opisujemy *kodem*, nie listą: zakresy, skala logarytmiczna, warunki („jeśli model to drzewo, to dobierz głębokość”).
- Opłaca się przy kosztownych modelach (sieci, boosting). Dla kNN wystarczy grid search.

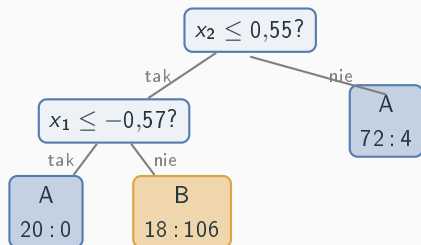
# Drzewa decyzyjne

---

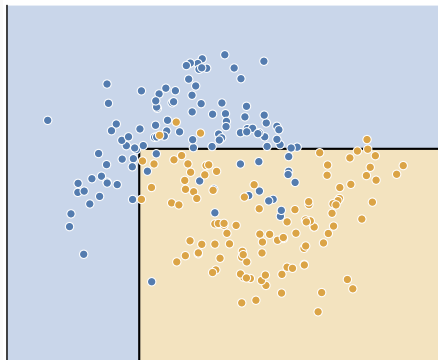
## Model, który wygląda jak if/else



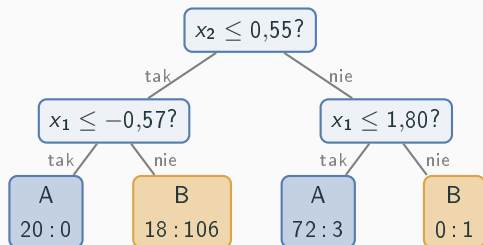
## Model, który wygląda jak if/else



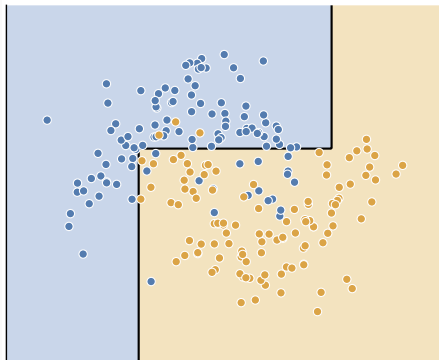
2 podziały: +  $x_1 \leq -0.57$  na dole



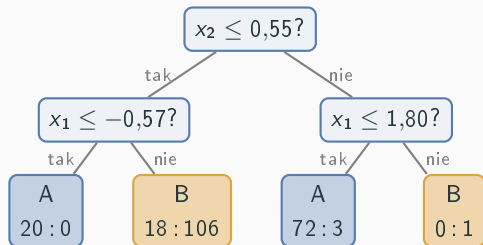
## Model, który wygląda jak if/else



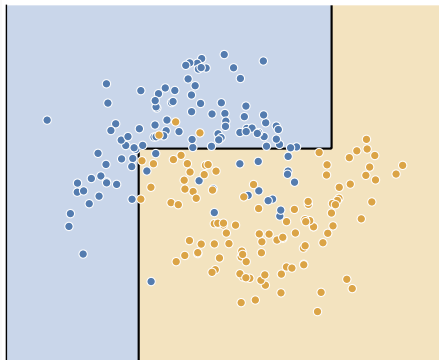
3 podziały: +  $x_1 \leq 1.80$  na górze



# Model, który wygląda jak if/else



3 podziały: +  $x_1 \leq 1.80$  na górze



- Każdy węzeł to pytanie o **jedną** cechę; każdy podział to cięcie **równoległe do osi**.
- Liść zwraca klasę większościową przykładów, które do niego trafiły (liczby w liściach: A : B).
- Nie wymaga skalowania cech, jest czytelne dla człowieka.

# Jak drzewo wybiera pytanie?

Miara „nieczystości” węzła z udziałami klas

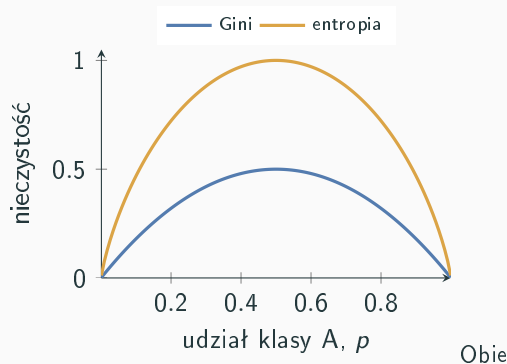
$p_1, \dots, p_C$ :

$$\text{Gini} = 1 - \sum_{c=1}^C p_c^2, \quad \text{entropia} = - \sum_{c=1}^C p_c \log_2 p_c$$

Zysk z podziału węzła  $t$  na  $t_L$ ,  $t_R$ :

$$\Delta = I(t) - \frac{n_L}{n} I(t_L) - \frac{n_R}{n} I(t_R)$$

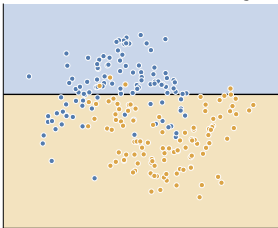
**Algorytm zachłanny:** w każdym węźle sprawdź każdą cechę i każdy próg, wybierz podział o największym  $\Delta$ , powtórz w dzieciach, aż liście będą czyste (lub osiągniemy limit).



miary są zerowe dla czystego węzła ( $p = 0$  lub  $p = 1$ ) i największe dla mieszanki pół na pół.

# Głębokość drzewa to regulator złożoności

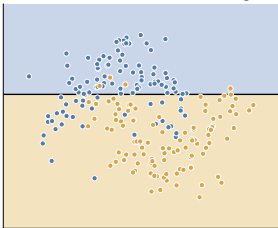
głębokość = 1 (trafność na treningu: 81%)



Jedno cięcie: niedouczenie.

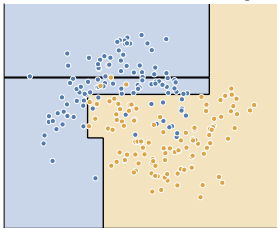
# Głębokość drzewa to regulator złożoności

głębokość = 1 (trafność na treningu: 81%)



Jedno cięcie: niedouczenie.

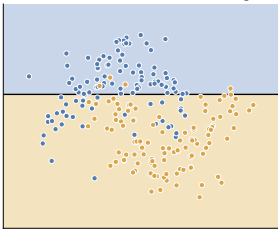
głębokość = 4 (trafność na treningu: 91%)



Kilka cięć: sensowny kształt.

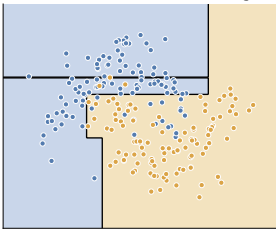
# Głębokość drzewa to regulator złożoności

głębokość = 1 (trafność na treningu: 81%)



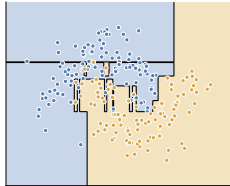
Jedno cięcie: niedouczenie.

głębokość = 4 (trafność na treningu: 91%)



Kilka cięć: sensowny kształt.

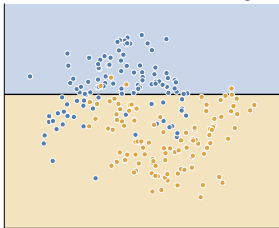
głębokość = bez limitu (trafność na treningu: 100%)



Bez limitu: własne pudełko dla każdego punktu szumu, 100 % na treningu.

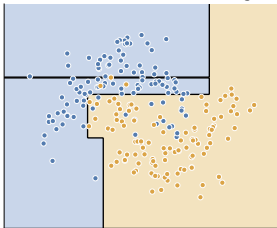
# Głębokość drzewa to regulator złożoności

głębokość = 1 (trafność na treningu: 81%)



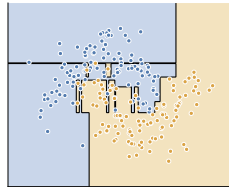
Jedno cięcie: niedouczenie.

głębokość = 4 (trafność na treningu: 91%)



Kilka cięć: sensowny kształt.

głębokość = bez limitu (trafność na treningu: 100%)



Bez limitu: własne pudełko dla każdego punktu szumu, 100 % na treningu.

## Hamulce

`max_depth`, `min_samples_leaf`, `min_samples_split` — dobierane walidacją. Drzewa mają **małe obciążenie i dużą wariancję**: mała zmiana danych daje zupełnie inne drzewo. To za chwilę wykorzystamy.

## Zespoły modeli

---

# Bagging: demokracja modeli

Każdy model ma jeden głos.

drzewo 1



drzewo 2



drzewo 3



drzewo 4

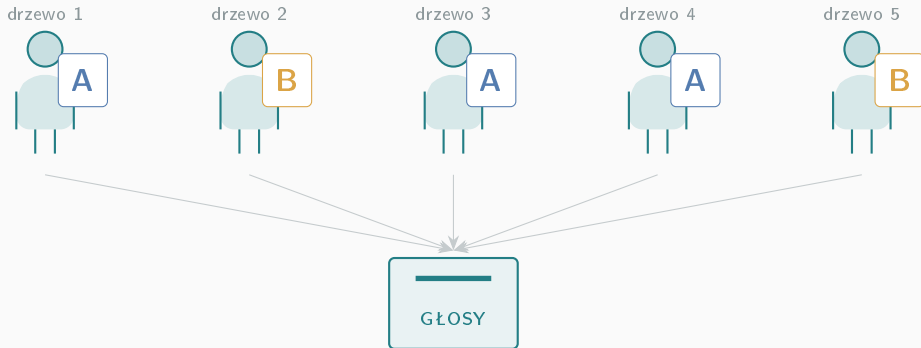


drzewo 5



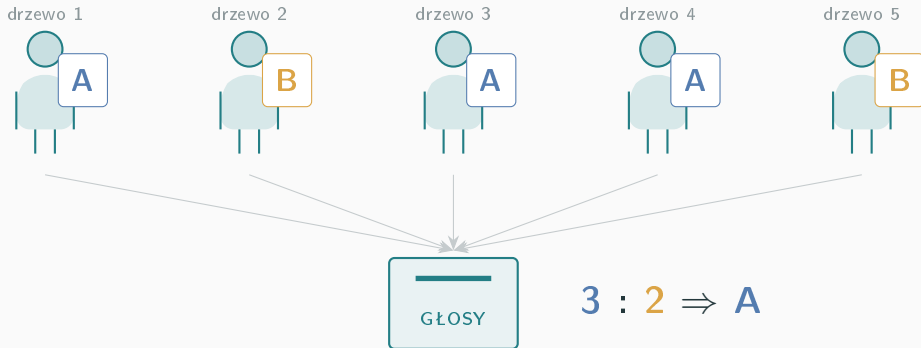
# Bagging: demokracja modeli

Każdy model ma jeden głos.



# Bagging: demokracja modeli

Każdy model ma jeden głos.



Pomaga różnorodność, nie kopiowanie tego samego modelu.

# Bagging: skąd różne głosy?

Dane

Losowanie ze zwracaniem

Drzewa

Decyzja

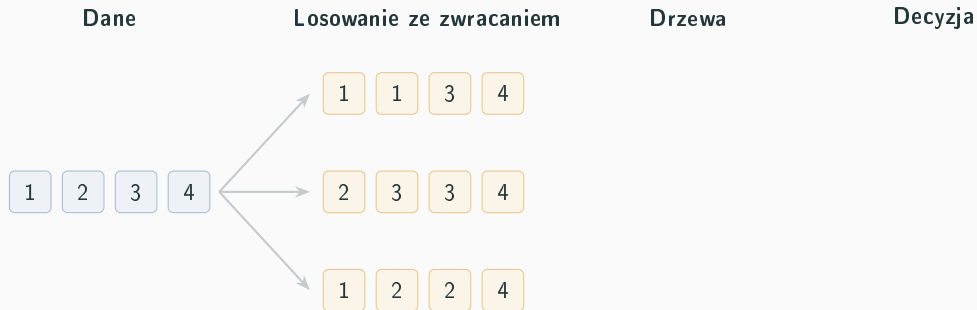
1

2

3

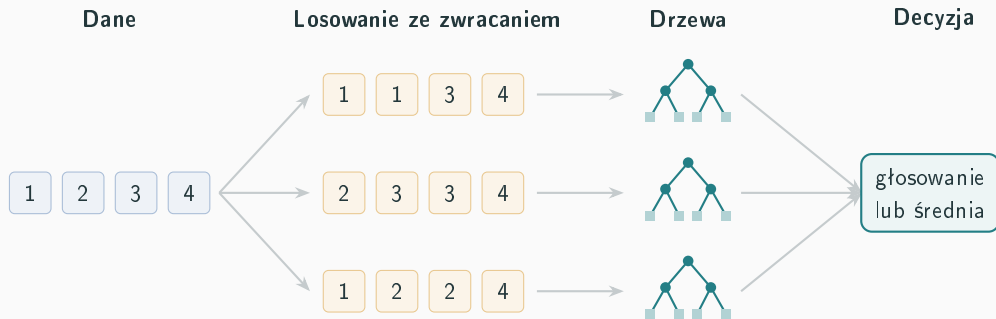
4

# Bagging: skąd różne głosy?



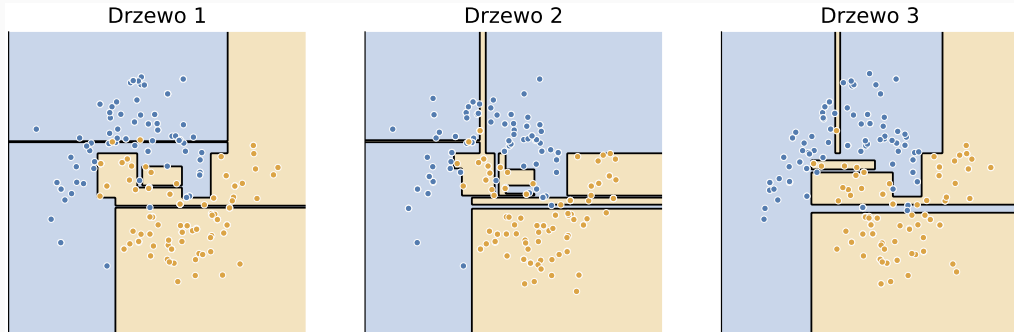
Inne próbki. Modele uczą się równoległe.

# Bagging: skąd różne głosy?



Inne próbki. Modele uczą się równoległe.

# Różne próbki, różne drzewa



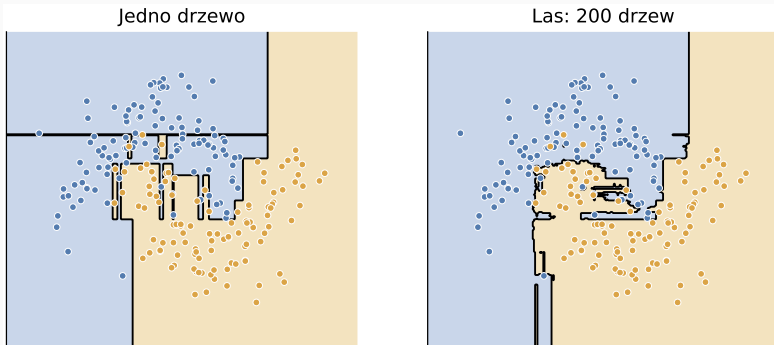
Każde drzewo myli się trochę inaczej.

# Las losowy: dwa źródła różnorodności

losowe próbki danych

+

losowe cechy przy każdym podziale



Wspólna decyzja jest mniej wrażliwa na pojedyncze drzewo.

# Voting: różne algorytmy, jedna decyzja

kNN

**A**

$$P(B) = 0,40$$



drzewo

**B**

$$P(B) = 0,90$$



regresja logistyczna

**A**

$$P(B) = 0,45$$



# Voting: różne algorytmy, jedna decyzja

kNN

**A**

$$P(B) = 0,40$$



drzewo

**B**

$$P(B) = 0,90$$



regresja logistyczna

**A**

$$P(B) = 0,45$$



Większość głosów

$A, B, A \Rightarrow A$

# Voting: różne algorytmy, jedna decyzja

kNN

**A**

$$P(B) = 0,40$$



Większość głosów

$A, B, A \Rightarrow A$

drzewo

**B**

$$P(B) = 0,90$$



Średnia prawdopodobieństw

$$\overline{P(B)} = 0,58 \Rightarrow B$$

regresja logistyczna

**A**

$$P(B) = 0,45$$



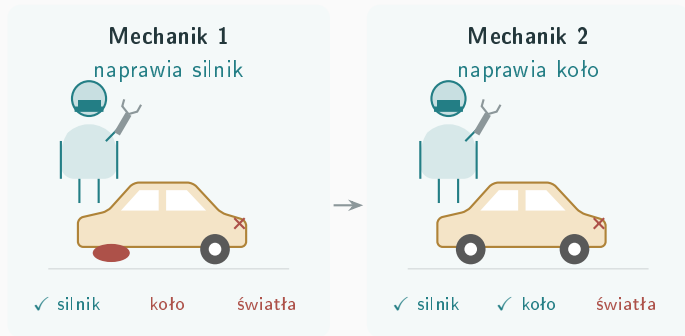
# Boosting: od mechanika do mechanika

Każdy naprawia to, co zostało.



# Boosting: od mechanika do mechanika

Każdy naprawia to, co zostało.



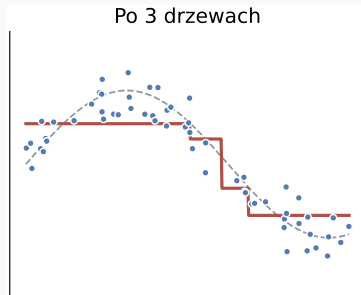
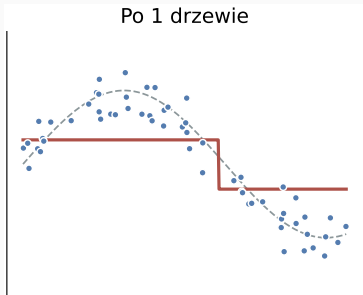
# Boosting: od mechanika do mechanika

Każdy naprawia to, co zostało.



Zespół zachowuje wszystkie wcześniejsze poprawki.

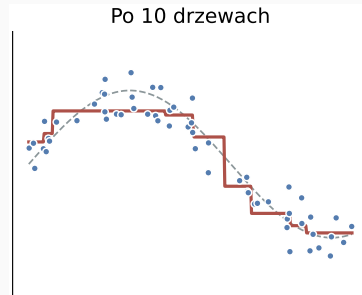
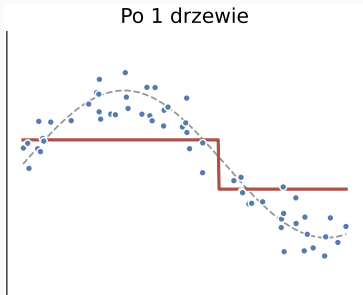
# Boosting: małe poprawki składają się na wynik



$$\underbrace{F_m(x)}_{\text{nowy wynik}} = \underbrace{F_{m-1}(x)}_{\text{dotychczasowy wynik}} + \underbrace{\eta h_m(x)}_{\text{mała poprawka}}$$

Przy błędzie kwadratowym poprawiamy reszty:  $y_i - F_{m-1}(x_i)$ .

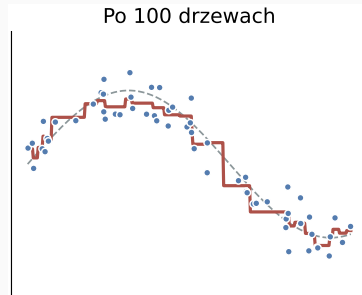
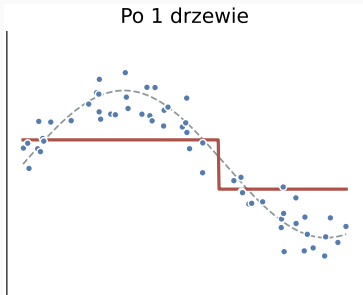
# Boosting: małe poprawki składają się na wynik



$$\underbrace{F_m(x)}_{\text{nowy wynik}} = \underbrace{F_{m-1}(x)}_{\text{dotychczasowy wynik}} + \underbrace{\eta h_m(x)}_{\text{mała poprawka}}$$

Przy błędzie kwadratowym poprawiamy reszty:  $y_i - F_{m-1}(x_i)$ .

# Boosting: małe poprawki składają się na wynik



$$\underbrace{F_m(x)}_{\text{nowy wynik}} = \underbrace{F_{m-1}(x)}_{\text{dotychczasowy wynik}} + \underbrace{\eta h_m(x)}_{\text{mała poprawka}}$$

Przy błędzie kwadratowym poprawiamy reszty:  $y_i - F_{m-1}(x_i)$ .

# Bagging i boosting: dwa sposoby współpracy

## Bagging



wspólny głos

Równoległe  
mniejsza wariancja

## Boosting



wynik + poprawka + poprawka

Po kolei  
korygowanie błędów

# XGBoost: boosting z hamulcami

Poprawiaj wynik. Pilnuj złożoności.

Płytke drzewa



Małe kroki



Kara za złożoność



Cel: błąd + kara za złożoność  $\rightarrow$  min

# XGBoost w kodzie

```
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=300,      # ile poprawek
    learning_rate=0.05,    # jak duży krok
    max_depth=4,           # jak złożona poprawka
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)])
```

liczba drzew

wielkość kroku

głębokość

Dobieraj razem na walidacji.

# Podsumowanie

---

# Co zabrać ze sobą

1. Liczy się wynik na danych, których model **nie widział**.
2. Trening uczy, walidacja wybiera, test ocenia — **raz**.
3. Przeuczenie = duża przerwa między treningiem a walidacją; niedouczenie = oba wyniki słabe.
4. Złożoność to regulator:  $k$ , stopień, głębokość, liczba drzew.
5. Walidacja krzyżowa: ten sam pomiar, mniej szczęścia.
6. Trafność kłamie przy rzadkich klasach; patrz na precyzję, czułość,  $F_1$ , AUC.
7. Próg decyzyjny to wybór, nie dogmat.
8. Hiperparametry: grid, random, Optuna — zawsze na walidacji.
9. Drzewo jest czytelne, ale niestabilne; las i boosting robią z tej wady zaletę.

## Jedno zdanie

Nie ma najlepszego modelu ani najlepszej metryki — jest tylko uczciwy pomiar dopasowany do problemu.

- Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning* (2009) — rozdz. 7 (ocena modeli), 9 i 10 (drzewa, boosting), 15 (lasy losowe)
- Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2019)
- Bishop, *Pattern Recognition and Machine Learning* (2006)
- Dokumentacja scikit-learn: [scikit-learn.org/stable/](https://scikit-learn.org/stable/) — działy *Model selection* i *Ensemble methods*
- Chen, Guestrin, *XGBoost: A Scalable Tree Boosting System* (KDD 2016)
- Akiba i in., *Optuna: A Next-generation Hyperparameter Optimization Framework* (KDD 2019)
- Bergstra, Bengio, *Random Search for Hyper-Parameter Optimization* (JMLR 2012)
- Materiały: [github.com/janchorowski/ml\\_uwr](https://github.com/janchorowski/ml_uwr)

Rysunki w tej prezentacji wygenerowano skryptem `scripts/make_figures.py` (numpy, matplotlib, scikit-learn); można je odtworzyć i modyfikować.